Модель на 35 миллиардов параметров на ноутбуке с 4 ГБ видеопамяти

Что мы надстроили над llama.cpp, как получили прирост скорости и что показало профилирование. Без магии: каждое число ниже измерено на этой же машине.

37 ток/с
генерация без лимита памяти (было 22; llama.cpp: 27)

86 ток/с
чтение промпта (llama.cpp: 49)

64K
окно контекста; кэш внимания занимает 0.73 ГиБ

22.9 ГБ
модель целиком на карте с 4 ГБ

> Прирост 22 → 37 ток/с получен программно, без разгона: частоты и профиль питания ноутбука не менялись.

0 Что такое Strata в целом, до нашего форка

Strata (проект Niko1221 на GitHub, лицензия MIT, открытый код) запускает на обычном игровом компьютере большие языковые модели, которым обычно нужен сервер. Главная модель апстрима — Qwen3.8-Flash-Next на 125 миллиардов параметров; с версии 0.2.4 в Терминале работает и DeepSeek V4.1-Flash. Есть и путь для модели поменьше, Qwen3.6-35B-A3B: именно он остался в нашем форке. Модель отвечает в чате, пишет код, читает картинки и работает с приложениями и кодовыми агентами. По README, чаты и файлы остаются на вашем ПК, аккаунт не нужен, облака и телеметрии нет (что и когда скачивается, описано в docs/SAFETY.md).

Как это устроено

Как если бы: библиотека. Модель — команда из 24 576 маленьких «специалистов» (экспертов), как 24 576 справочников; на каждое слово нужны только 10. Видеокарта — рабочий стол: маленький и быстрый, на нём открыты примерно 4 000 самых частых книг. Оперативная память — полки читального

зала: там лежат все 24 576, и когда нужной книги нет на столе, процессор берёт её с полки сразу, пока карта продолжает работу. SSD — архив: там таблица в 29 ГБ, из которой на слово читается несколько строк, поэтому в память её держать не нужно. Длинные тексты читаются большими кусками, до 8192 токенов за раз, быстрее 1000 токенов в секунду.



Что лежит в репозитории

Каталог (файлов)	Что это
apps (822)	настольное приложение на Node: чат, монитор GPU/CPU/ОЗУ, страница моделей, настройки; агент работает в папке на ПК и в режиме Manual спрашивает перед изменением файла или запуском команды
src, include (313 + 174), metal, sycl (96 + 251)	движок на C++ для CUDA и HIP; каталоги metal и sycl по названиям относятся к Mac и Intel
serve (108)	HTTP-сервер на Python: API OpenAI, Anthropic и Responses, шаблон чата, инструменты, метрики
strata_control, strata_cli (61 + 37)	Терминал: команда strata (launch, monitor, update, doctor, connect)
packaging (30), setup.sh, START-HERE.bat, Docker	установщики и сборка пакетов; MCP-сервер, через который ИИ-ассистенты ставят и запускают Strata
tools, bench, tests (317 + 2229 + 209)	скрипты сборки, бенчмарки с данными, тесты
docs (135), data (21)	документация: модели, AMD, Intel, Mac, как это работает, безопасность, устранение неполадок

Всего 4892 файла. Скорость и ядра берутся из llama.cpp / ggml (часть кода и все модели имеют свои лицензии, список в docs/HOW_IT_WORKS.md). Модели сжали ISTA-DASLab, UkisAI (Swift 1.5), agentionai (Gyro) и Unsloth.

Какие модели и сколько памяти нужно

Минимум 32 ГБ памяти, считая ОЗУ и видеопамять вместе (на Mac и Ryzen AI Max это единая память), и около 80 ГБ свободного SSD. Поддерживаются NVIDIA GeForce RTX 20–50 (рекомендуется 12 ГБ и больше), AMD Radeon RX 6800 и новее, Radeon AI PRO R9700, Intel Arc, Apple Silicon (M1 и новее, macOS 15), Ryzen AI Max, Windows 10/11, Linux и macOS; две-три карты одной марки могут делить модель. Размер подбирается под вашу память:

Ваша память (ОЗУ + VRAM)	Что брать (Qwen3.8-Flash-Next)
32 ГБ	Gyro-S (27.6 ГБ памяти, 58.5 ГБ загрузки) или Coder для кода
40 ГБ	Swift 1.5 IQ2_XS (39.2 ГБ) или Gyro-M (35 ГБ памяти)
48 ГБ	IQ3_XXS (47.0 ГБ)
64 ГБ	IQ3_S (54.8 ГБ): по опубликованным тестам равна полной модели
96 ГБ и больше	Unsloth UD-IQ4_XS … Q8_0

Другие модели: DeepSeek V4.1-Flash запускается на одной карте в 12 ГБ при 46–64 ГБ ОЗУ, самое занятое держится в памяти, остальное читается с SSD (наборы 239–274 ГБ). Qwen3.6-35B-A3B (35 миллиардов параметров, 3 миллиарда активных, 256 экспертов, по 8 на слово) в апстриме описана как модель поменьше, которая читает картинки и обслуживает несколько запросов сразу; её замеряли на Radeon AI PRO R9700 32 ГБ и RTX 3060 12 ГБ, где 22–37 экспертов на слой лежат в видеопамяти, а остальные считает процессор. Скорость, по замерам автора на RTX 5070 12 ГБ при 64 ГБ ОЗУ: Q2_0 отвечает на 94 ток/с и читает промпт на 2650 ток/с, IQ3_S — на 53 и 1620 ток/с. Первое сообщение чата читается целиком, около минуты на 30 000 токенов.

Для кого

Кто вы	С чего начать
Хотите обычный чат на своём ПК, без облака	приложение для Windows, Linux или Mac: скачать, открыть, нажать «установить» на рекомендованной модели
Разработчик с кодовыми агентами	Терминал: strata launch; strata connect показывает настройки для Claude Code, Codex CLI, OpenCode, pi, Hermes Agent и OpenClaw; адрес http://127.0.0.1:8080 (OpenAI, Anthropic, Responses)
Скрипты, серверы и ИИ-ассистенты	strata launch --yes --no-tui, docs/AI_SETUP.md и MCP-сервер
Кому не подходит	машинам с суммарной памятью меньше 32 ГБ и картам меньше 12 ГБ: вне рекомендаций README; наш ноутбук (4 ГБ видеопамяти) как раз такой

Что показала проверка безопасности

Мы проверили апстрим 24 октября 2026 (коммит 7dd5cf18, релиз v0.2.4): признаков вредоносного кода в установщиках, цепочке обновлений, Терминале и бинарниках движка нет, а собранные файлы совпадают с подписанным каналом обновлений. Найдены слабые места, не злонамеренные: шлюз подтверждений в настольном агенте пропускает sort -o, uniq IN OUT, tree -o и rg --pre CMD без вопроса; API хоста на Node без секрета и с CORS *; Python-сервер скачивает картинки по URL без фильтра частных адресов; релизы собираются вручную, без CI; install.sh проверяет подпись бинарником и ключом из того же архива. Скорость на нашем ноутбуке тогда не мерили.

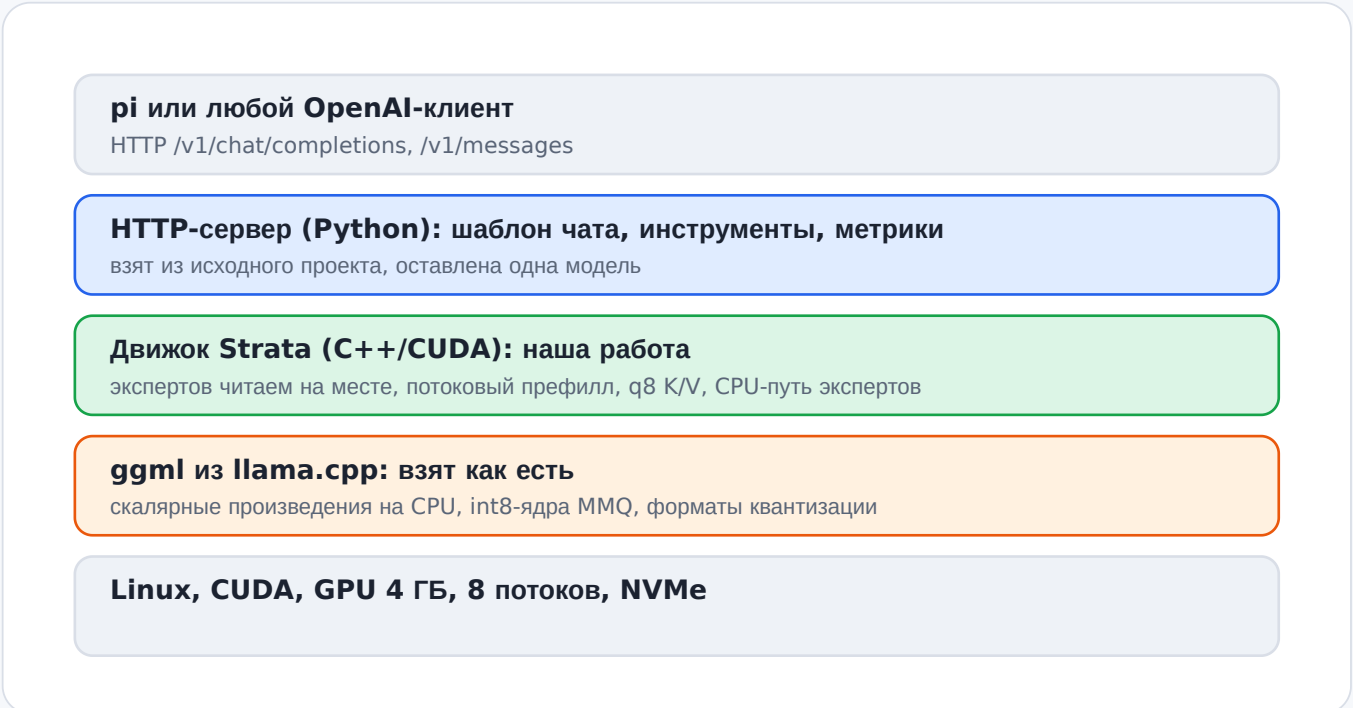
Что мы взяли, а что нет

Из 4892 файлов в форке остались движок для Qwen3.6-35B-A3B, сервер, часть Терминала, инструменты, тесты и нужная часть ggml. Настольное приложение с агентом, установщики, бенчмарки с данными и другие движки убраны, поэтому слабые места шлюза и CORS в форк не вошли. Дальше рассказано, что мы надстроили для карты на 4 ГБ.

1 Что это и на чём построено

Модель Qwen3.6-35B-A3B — «смесь экспертов»: в каждом из 40 слоёв 256 небольших специалистов, но на каждый токен работают только 8 из них. Поэтому активных параметров всего около 3 миллиардов, и модель можно запустить на слабом железе, если умно развести данные между видеокартой, процессором и диском.

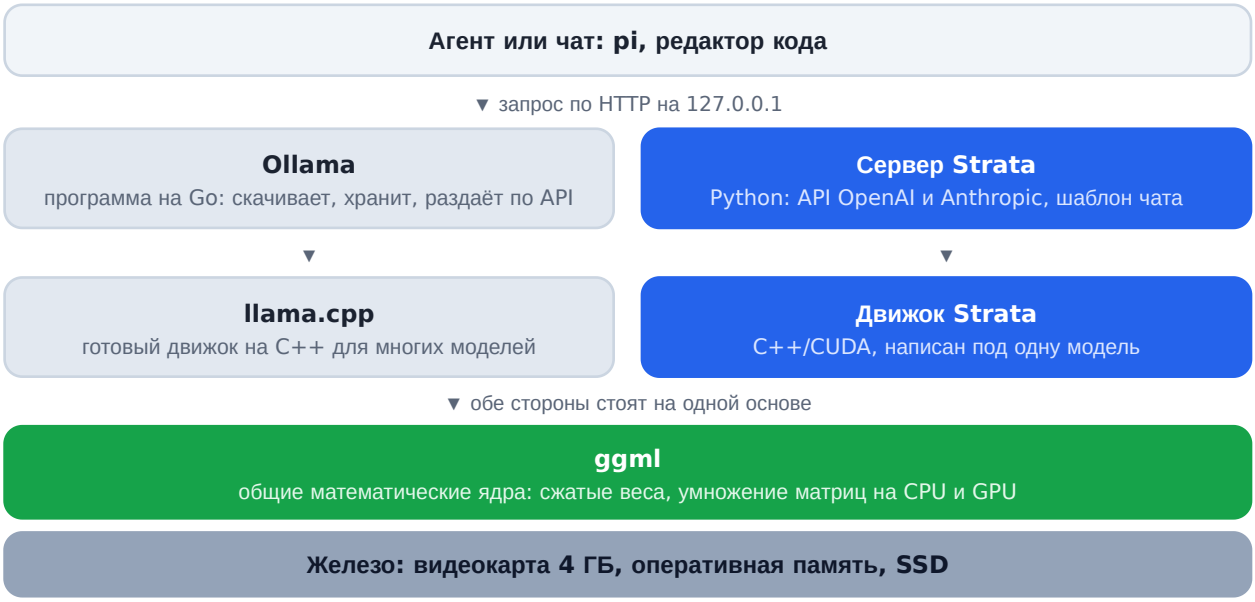
Основа — открытые ядра ggml из llama.cpp (квантизация, скалярные произведения на CPU, int8-умножение матриц на GPU). Поверх них стоит движок Strata, из которого мы выбросили всё лишнее (4892 файла → около 230, 167 МБ → 16 МБ) и дописали то, что нужно именно этой машине. Сравниваем мы с самим llama.cpp (версия от 25.10.2026): Ollama в замерах не участвовала.



1.1 Кто есть кто: ggml, llama.cpp, Ollama и Strata

коротко, в пяти предложениях

Ни «ollama.cpp», ни «strata.cpp» в этом стеке нет: есть **llama.cpp** — готовый движок на C++, который читает файл модели (GGUF) и считает ответ по одному токену, и есть **Ollama** — программа-обёртка на Go, которая скачивает модели и раздаёт их по API, а считает в основном встроенным llama.cpp. Под llama.cpp, а значит и под Ollama, лежит библиотека **ggml** — набор математических «кирпичиков»: сжатые веса, умножение матриц на CPU и GPU. **Strata** — не расширение и не контроллер для Ollama: она не вызывает ни Ollama, ни llama.cpp как программу, а берёт из llama.cpp только ядра ggml и пишет поверх них собственный движок. Этот движок рассчитан на одну модель, Qwen3.6-35B-A3B, и сам раскладывает её веса по видеокарте, оперативной памяти и SSD, поэтому на этом ноутбуке он быстрее общего llama.cpp: чтение длинного промпта 86 против 49 ток/с, генерация быстрее на 5–20% (раздел 9). Снаружи Strata выглядит как Ollama — локальный сервер с API OpenAI и Anthropic на `127.0.0.1:8080`, к которому подключается агент pi, — а «strata.cpp» это просто разговорное имя C++/CUDA-части её движка.



Название	Что это простыми словами	Какое отношение к **Strata**
ggml	Библиотека математики на C: хранит веса в сжатом виде, умножает матрицы на CPU и GPU.	Берём готовые ядра из дерева llama.cpp (`third_party/`).
llama.cpp	Готовый движок и сервер на ggml: читает файл модели GGUF и считает ответ токен за токеном.	Как программа не используется: из него взяты только ядра ggml. Служит эталоном в замерах (раздел 9).
Ollama	Удобная оболочка на Go: скачивает модели одной командой, хранит их, поднимает локальный сервер; токены считает в основном встроенным llama.cpp.	Не используется и в замерах не участвовала.
Strata	Свой движок и сервер под одну модель: сам раскладывает веса по видеокарте, оперативной памяти и SSD.	Это наш проект (форк Niko1221/Strata, лицензия MIT).
«strata.cpp»	Разговорное имя C++/CUDA-части движка (каталоги `src/` и `include/`).	Отдельного проекта или файла с таким именем в репозитории нет.

Как если бы: ggml — детали двигателя; llama.cpp — готовый универсальный автомобиль из этих деталей; Ollama — сервис с приложением и водителем вокруг такого автомобиля; Strata — гоночная машина под одну трассу, собранная из тех же деталей.

Частые вопросы

Это расширение или контроллер для Ollama?
Нет. Strata — отдельная программа рядом с Ollama: у неё свой сервер и свой движок, Ollama она не запускает и не вызывает. Общее у них одно — математические ядра ggml.

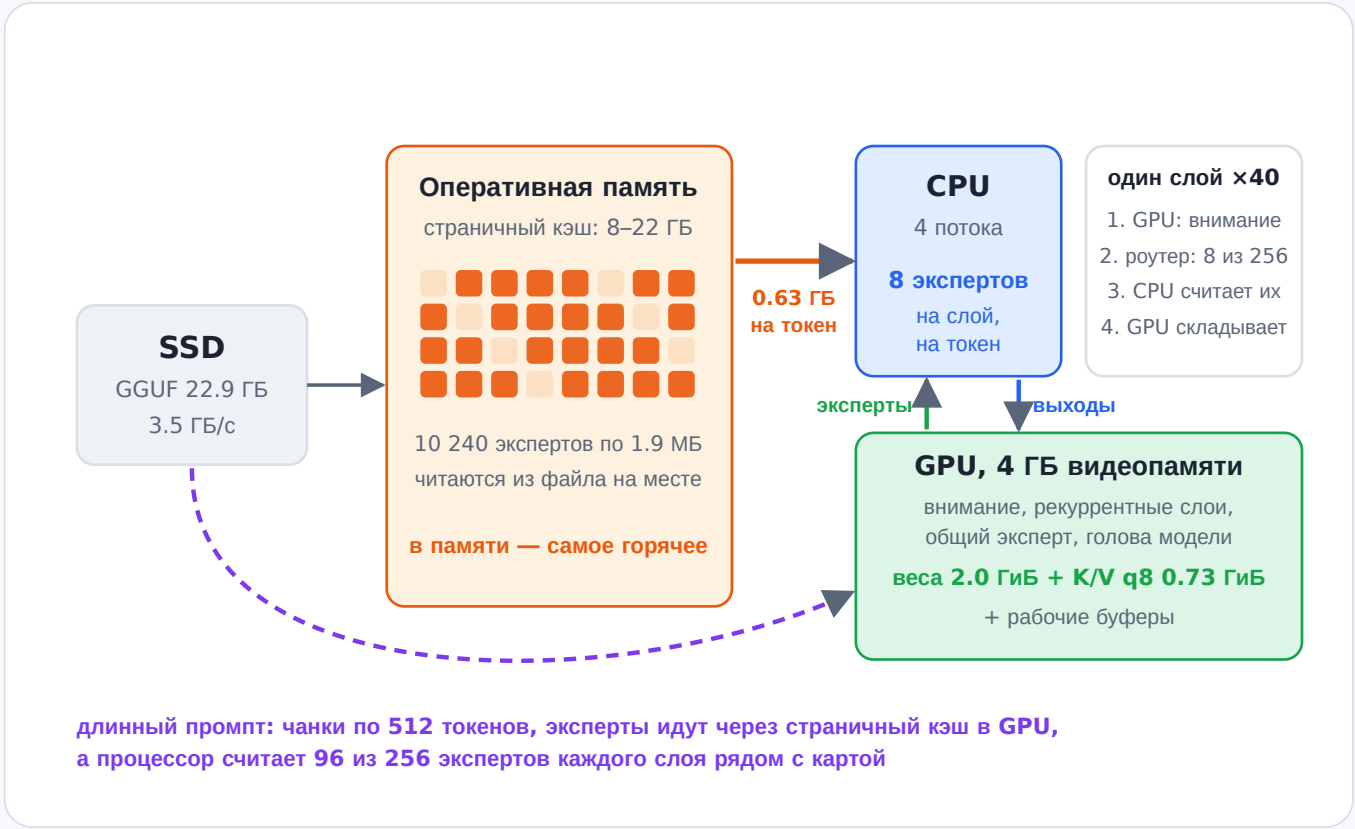
Нужны ли Ollama или llama.cpp, чтобы запустить Strata?
Нет. ggml входит в сборку (`third_party/`), а файл модели скачивает `tools/fetch_model.sh` с проверкой размера и SHA-256.

Что происходит с одним вопросом?
Агент шлёт запрос на `127.0.0.1:8080` ; Python-сервер собирает промпт по шаблону чата и запускает движок отдельным процессом; движок считает слои: видеокарта — внимание, процессор — нужные 8 экспертов прямо из файла; ответ возвращается агенту.

Почему сравнивают с llama.cpp, а не с Ollama?
Токены считает именно llama.cpp, в том числе внутри Ollama. Ollama в замерах не участвовала.

2 Как 22 ГБ живут на карте в 4 ГБ

Видеокарте достаётся «плотная» часть — внимание, рекуррентные слои, общий эксперт (2.1 ГиБ): её нужно читать на каждом токене, и у GPU для этого быстрая память. Тысячи «редких» экспертов остаются в файле на диске; процессор считает нужные 8 прямо из страничного кэша, ничего не копируя. Что не поместилось в кэш — подтягивается с SSD.



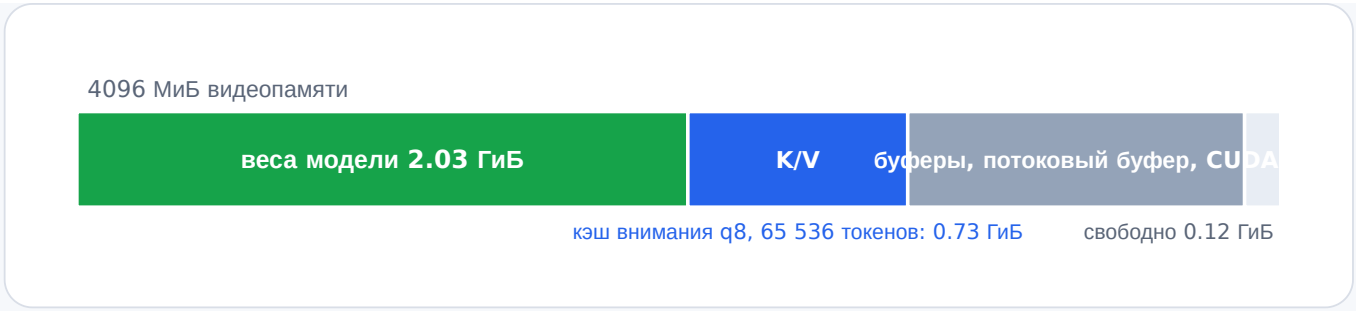
На каждом слое GPU и CPU работают по очереди: GPU считает внимание и выбирает экспертов, CPU считает их, GPU продолжает. Чтобы это окупилось, ускорены обе половины и сам обмен между ними (раздел 4).

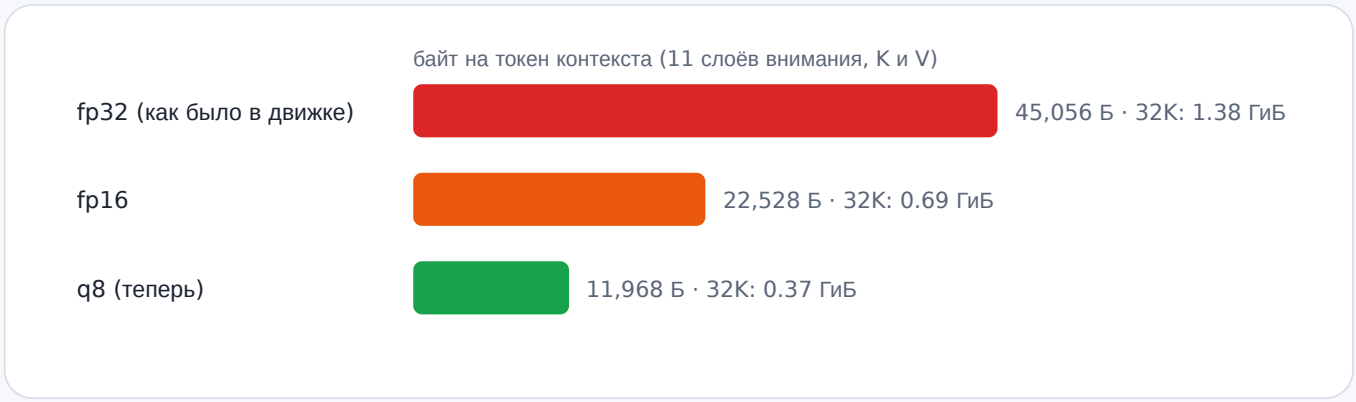
Подсказки вперёд: черновики MTP

В модель встроен маленький блок MTP: он угадывает два следующих токена. Основная модель проверяет все три за один проход, и около 90% догадок оказываются верными, так что один проход даёт в среднем 2.8 токена. Это работает только для «жадной» генерации (без температуры и штрафов).

3 Память: где каждый мегабайт

Видеопамять — самый дефицитный ресурс. Кэш внимания (K/V) хранит прошлое разговора; он растёт с длиной контекста. Мы перевели его из 32-битных чисел в 8-битные блоки (формат q8_0 из ggml): почти вдвое меньше fp16 и в 3.8 раза меньше исходного fp32 — токены при этом совпадают с эталоном.

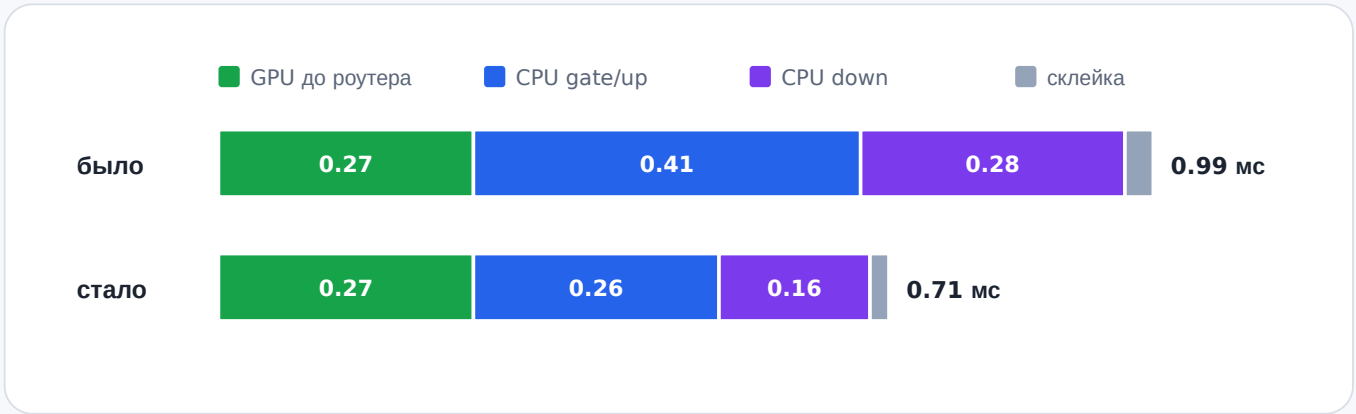




При окне 64K и чанке 512 в карте остаётся 0.12 ГиБ (при окне 32K — 0.49 ГиБ; до правок раздела 10 при 32K было 0.14). Чанк 768 при окне 32K влезал (0.21 ГиБ до объединения буферов) и читал промпт в 3099 токенов за 15.5 с против 18.3 с на чанке 512 в той же серии; при окне 64K его не пробовали. Размер чанка меняет границы порций, поэтому в редких почти равных случаях вывод может отличаться: по умолчанию остаётся 512.

4 Профилирование: куда уходило время

Инструменты: perf (где процессор тратит такты), nsys (временная шкала GPU), счётчики внутри движка и два микробенчмарка — пропускная способность памяти и скорость скалярного произведения Q4_K. Один токен = 40 слоёв, и в каждом слое время делится так:



находка 1

Подсказка ядру стоила дороже работы

Перед каждым слоем движок просил систему заранее подгрузить 24 диапазона экспертов (madvise WILLNEED). Даже когда всё уже в кэше, система обходит каждую страницу: сотни микросекунд впустую. Теперь подсказка идёт, только если средняя страница диапазона не в памяти.

+17% на тёплом кэше

находка 2

Память можно читать быстрее

Микробенчмарк: четыре потока читают 54 ГБ/с, а скалярное произведение Q4_K выдавало 40 ГБ/с. Аппаратный предзагрузчик процессора «забывается» на границе каждой страницы 4 КБ. Мы добавили явную предзагрузку на три строки вперёд в циклах экспертов.

время CPU-пути экспертов −37…41%

находка 3

Черновики MTP были выключены

Сервер добавлял к запросам «рекомендованную» выборку с температурой и штрафом за повтор, а черновики работают только на жадных запросах. Для агента в консоли теперь по умолчанию жадная генерация (флаг --sampling card возвращает прежнее).

+6…26% на реальном тексте

находка 4

Потолок GPU-части

Одно ядро (плотные матрично-векторные произведения) занимает 58% времени GPU: на токен читаются 2.1 ГиБ весов, то есть ≈ 11–12 мс при полной скорости видеопамяти. Это уже близко к физике, дальше выигрыш только за счёт сжатия весов.

остаётся ≈ 0.27 мс на слой

находка 5

Длинный промпт упирался в диск

Каждый проход по чанку читает все эксперты один раз, поэтому скорость промпта растёт вместе с чанком, а видеопамять ограничивает его размер. Читаем O_DIRECT-потоком, чтобы промпт не вытеснял горячие эксперты из кэша.

86 ток/с против 49 у llama.cpp

находка 6

Ноутбук греется за 10 секунд

Процессор достигает 100 °C примерно за 10 секунд генерации, частота скачет между 3.2 и 4.3 ГГц. Поэтому короткий запрос может показать 37 ток/с, а тот же запрос через минуту нагрузки — 25–28. Мы не разгоняли и не меняли питание, а замеряли только с одного и того же теплового состояния.

честный замер: старт ≤ 55 °C, 900 токенов



5 Потолок: почему не 60 токенов в секунду

Для 60 ток/с один шаг генерации (проверка 3 токенов, в среднем 2.7 принятых) должен уложиться в 45 мс. Сейчас он занимает 75 мс. Ниже — куда уходят эти миллисекунды и где физический предел именно этой машины.



Арифметика: откуда берутся 36 и 68 ток/с

Скорость — это число токенов за шаг, делённое на время шага: $2.7 \div \text{время шага}$. Исходные данные — счётчики прогона на 400 токенов при холодной машине и модели целиком в кеше страниц.

Случай	Время шага	Скорость
Холодная машина, измерено	`14.9 + 52.0 + 8.5 = 75.4 мс`	`2.7 ÷ 0.0754` ≈ **36 ток/с** (измерено 35.6)
Разогретый CPU	`14.9 + 52.0 × 1.51 + 8.5 ≈ 102 мс`	`2.7 ÷ 0.102` ≈ **26.5 ток/с** (измерено 25.9)

Потолок по памяти	`GPU: 2.27 ГБ ÷ 172 ГБ/с = 13.2 мс`
	`CPU: ≈ 1.4 ГБ ÷ 54 ГБ/с → 26.1 мс` (по счётчику)
	`13.2 + 26.1 = 39.3 мс`
		`2.7 ÷ 0.0393` ≈ **68.7 ток/с**

×5 к 36 ток/с	`2.7 ÷ 180 = 15 мс`	`180 ток/с` — в 2.6 раза выше потолка

Откуда числа: 2.7 — токена за шаг (84% черновиков принимаются); 14.9 — GPU, плотные веса; 52.0 — CPU, эксперты; 8.5 — черновик, сэмплинг, склейка; 1.51 — рост CPU-части на горячей машине (7.7 с → 11.7 с), GPU-часть не меняется. Времена GPU и CPU складываются, а не перекрываются: слой L+1 ждёт выход слоя L.

> **Что значит ×5.** От 36 ток/с это ≈ 180 ток/с, то есть шаг ≈ 15 мс. Одна лишь память CPU-части даёт 26 мс, а память GPU-части — 13.2 мс. Даже при бесплатной CPU-части GPU-часть ограничивает скорость числом `2.7 ÷ 0.0132 ≈ 205 ток/с`. Значит, ×5 достигается не быстрее написанным кодом, а меньшим объёмом весов, который нужно прочитать за шаг, или более быстрой памятью.

узкое место 1

CPU читает эксперты на половине скорости памяти

На шаг приходится ≈ 1.4 ГБ весов экспертов. Память отдаёт 54 ГБ/с, то есть минимум 26 мс; на деле уходит 52 мс (≈ 29 ГБ/с). Причины: на каждый вес приходится 1.4 токена (расчёт тратит такты), два барьера на слой, процессор греется.

потолок 26 мс · есть 52 мс

узкое место 2

Видеопамять уже на пределе

Каждый шаг читает все плотные веса (2.27 ГБ). Микробенчмарк чистого чтения: карта отдаёт 172 ГБ/с, значит минимум 13.2 мс. Мы тратим 14.9 мс. Здесь выжимать почти нечего, кроме сжатия самих весов.

потолок 13.2 мс · есть 14.9 мс

узкое место 3

GPU и CPU ходят по очереди

Слой L+1 нужен выход слоя L, поэтому GPU ждёт CPU, а CPU ждёт GPU: времена складываются (15 + 52), а не перекрываются. Даже на потолках памяти получилось бы 39 мс, то есть около 68 ток/с — и это без единой потери на склейку.

теоретический предел ≈ 68 ток/с

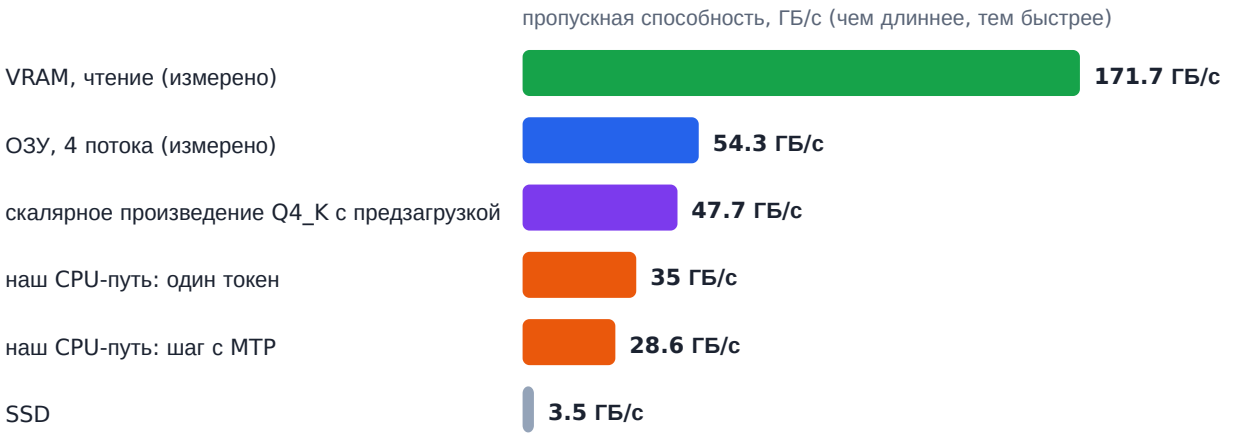
узкое место 4

Нагрев замедляет только CPU-часть

Два одинаковых прогона подряд: GPU-часть заняла 2.2 с оба раза, CPU-часть — 7.7 с и 11.7 с (+51%). Скорость упала с 35.6 до 25.9 ток/с. Это свойство машины; мы его не трогали и не разгоняли.

CPU +51%, GPU +0%





Что с этим можно сделать

Идея	Что даёт	Оценка	Статус
Больше памяти под страничный кэш	меньше обращений к SSD	измерено: 10 → 33 ток/с	готово (профили)
Один барьер на слой: каждый поток считает эксперта целиком (gate/up → down)	меньше простоя и дисбаланса	оценка: +5…10%	не сделано

Идея	Что даёт	Оценка	Статус
Ядра AVX-512 VNNI, где вес декодируется один раз на 2–3 токена	меньше тактов на байт при проверке нескольких токенов	оценка: +10…20%	не сделано
Перекрыть GPU и CPU (черновик следующего шага на GPU, пока CPU проверяет)	сумма 15 + 52 мс стремится к максимуму	оценка: до +15%, сложно	исследовать
Эксперты в 3-битном формате (другой файл модели)	на 25–30% меньше байт на шаг	оценка: +20… 30%, но качество меняется	нужно ваше решение
Плотные веса GPU в 6 бит вместо 8	GPU-часть читает на 25% меньше	оценка: +5%, качество меняется	нужно ваше решение
Железо: память быстрее, чем 54 ГБ/с, или GPU с 12+ ГБ (эксперты в видеопамяти)	сдвигает сами потолки	кратно	не для этого ноутбука

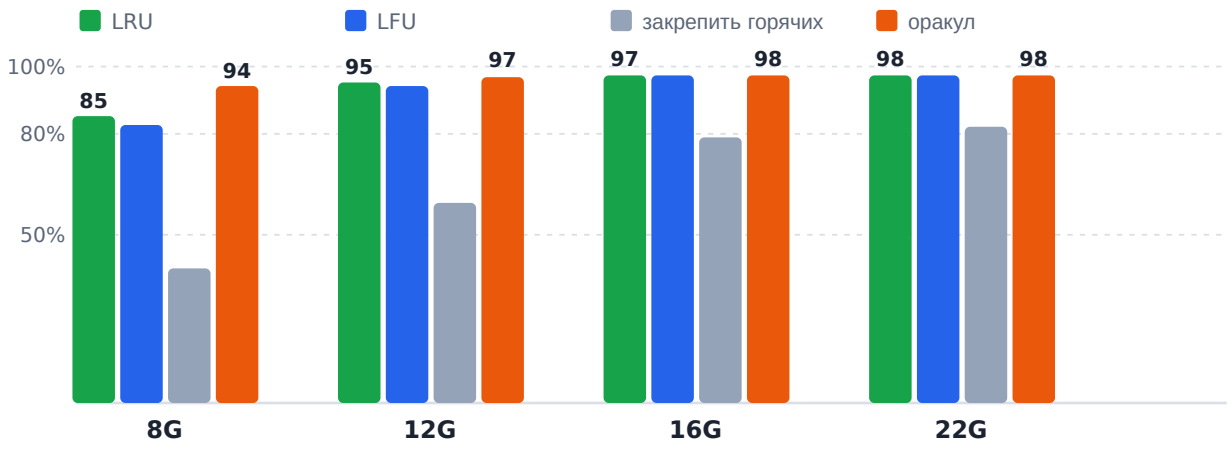
> Оценки в таблице — расчёт по измеренным потолкам, а не замер: их стоит проверять по одной, как мы проверяли предзагрузку.

6 Проверка идей

Вы предложили много направлений. Ниже каждое из них проверено: замером на этой машине, там где это возможно, иначе по документации и найденным исследованиям (это помечено). Значки: ✔ работает, ~ есть смысл, но нужна работа или ваше решение, ✘ не поможет.

1. Другой алгоритм кэша, чтобы чаще попадать

Представьте кухню и склад: на столе помещается только часть продуктов, остальное в кладовке за дверью. Алгоритм кэша решает, что держать на столе. Мы записали, какие эксперты нужны модели на реальном тексте, и прогнали эту запись через разные «правила стола», включая идеального «оракула», который заранее знает будущее.



доля обращений к экспертам, найденных в памяти; два текста, 600 и 500 токенов, холодный старт включён

Вывод: обычное правило «убрать то, что давно не трогали» (LRU), которое уже работает в системе, почти не уступает оракулу при 12 ГБ и выше. Даже идеальный алгоритм прибавил бы лишь 9 пунктов при 8 ГБ, 2 при 12 ГБ и почти ничего при 16 ГБ. «Закрепить самых популярных» работает хуже: у разных текстов разные любимцы. Значит, дело не в алгоритме, а в том, чтобы вовремя подвозить недостающее.

2. Подвозить заранее: предсказание следующего слоя

Как официант, который накрывает следующий стол, пока вы доедаете. Модель выбирает экспертов слой за слоем; если угадать выбор следующего слоя, пока считается текущий, диск успеет подвезти недостающее. Но у диска своя ширина трубы: за время одного слоя он успевает подтянуть около 1.3 эксперта.



Что проверено: простой статистический предсказатель (по тому, какие эксперты шли вместе на прошлом тексте) угадывает лишь 14% выбора следующего слоя (случайно было бы 3%), так что нужен настоящий роутер следующего слоя на текущем скрытом состоянии. В найденных исследованиях такой подход даёт 90%+ точности на других моделях (в источниках не проверено мной). Большая оговорка: при 8 ГБ промахов столько, сколько диск вообще способен подвезти, то есть выигрыш там ограничен самим диском. Затем я реализовала это в движке и замерила при 12 ГБ: точность выросла до 77–95% (при 8, 12 и 20 кандидатах), а скорость нет: 14.1 ток/с при 2–3 кандидатах и 10.7 при 12 против 14.4 без предзагрузки. Вероятная причина (не проверена): при жёстком лимите каждая подгрузка требует вытеснять другие страницы, и это стоит процессорного времени; лишние угаданные эксперты ещё и забивают диск. Код убран.

✘ Сначала считать то, что уже в памяти

Как если бы: сначала обслужить гостей, чей заказ уже готов, пока повар доготавливает остальное

Реализовано и замерено при 12 ГБ: 14.05 против 14.65 ток/с без перестановки (по два запуска). Подгрузка с диска ждёт дольше, чем считаются уже готовые эксперты.

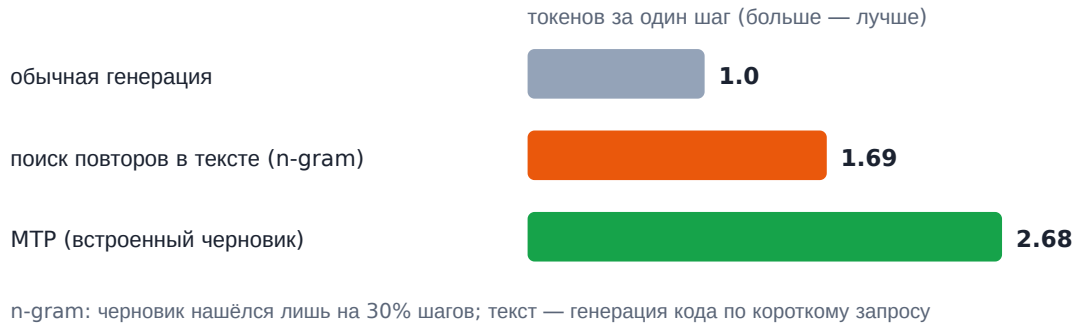
Не помогло, откатили.

✘ Вспомогательная модель-черновик

Как если бы: стажёр пишет черновик, профессор проверяет

Маленькая плотная модель (0.6–1.7 млрд) не влезает: свободно 143 МБ видеопамяти, и она делила бы с экспертами пропускную способность ОЗУ. Голов EAGLE-3 для Qwen3.6 в найденных источниках нет. Поиск повторов в тексте (n-gram, нулевая память) замерен на реальной генерации: 1.69 токена за шаг против 2.68 у встроенного MTP.

MTP уже лучший помощник; n-gram может пригодиться при правке длинных файлов (не замерено).



n-gram: черновик нашёлся лишь на 30% шагов; текст — генерация кода по короткому запросу

✘ Эффективнее использовать видеопамять

Как если бы: чемодан, в который не влезет шкаф: можно уложить аккуратнее, но шкафом он не станет

Видеопамять уже заполнена весами (2.16 ГиБ), кэшем внимания (0.37) и рабочими буферами. Держать в ней экспертов бессмысленно: в 143 МБ помещается 75 экспертов, а они закрывают 0.8% обращений (график ниже). Остаётся лишь сжать голову модели (515 МиБ) в 4 бита и освободить около 260 МБ, но это меняет ответы.

Выжимать нечего.

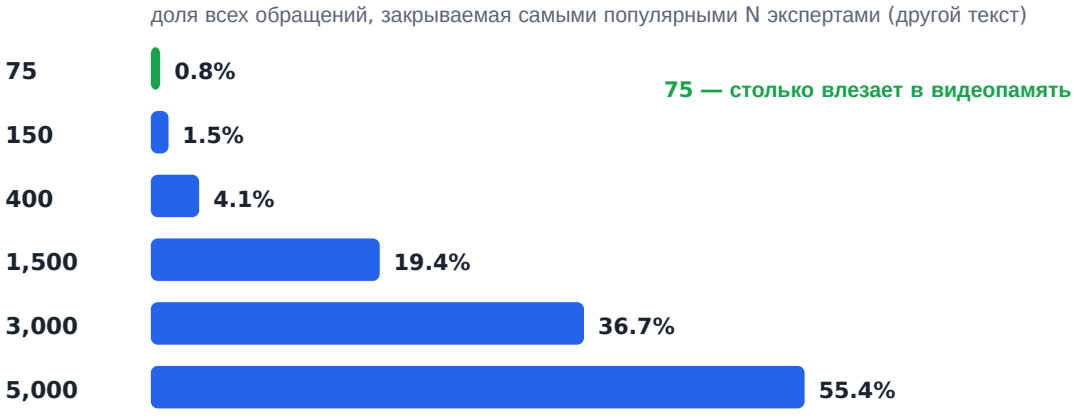
✘ Всё нужное поместить в видеопамять

Как если бы: переехать в квартиру размером с чемодан

За 600 токенов одного текста понадобилось 8 458 из 10 240 экспертов (≈ 16 ГБ). «Нужное» не небольшой набор: это почти вся модель. Вариант один: другая, заметно меньшая модель, а это другое качество и ваше решение.

Невозможно с этой моделью.





3. Диск, шифрование и «второй источник»

✘ Видеокарта читает SSD напрямую

Как если бы: второй курьер к тому же складу: склад один, дверь одна

По документации NVIDIA (GPUDirect Storage) прямой режим есть только у карт для дата-центров и Quadro, у остальных работает режим совместимости, который всё равно идёт через оперативную память. В системе модуль nvidia-fs не загружен, а файл лежит на ext4 поверх шифрования LUKS. Главное: ядро показывает, что SSD (Samsung 990 PRO, умеет Gen4) обучил линк на PCIe Gen3 x4, а это ровно измеренные 3.4 ГБ/с. Вторая дорога не расширит ту же трубу.

Выигрыша нет.

~ Шифрование диска стоит процессора

Как если бы: охранник на складе проверяет каждую коробку

Том зашифрован (LUKS2, AES-XTS 512 бит, секторы по 512 байт, без флагов ускорения). Чтение модели с диска занимает около 1.6 ядра из 8 (в основном расшифровка). Это не ограничивает скорость (ограничивает линк), но каждый промах кэша греет процессор. Сырое устройство против LUKS не измерено: проба требовала пароль, который не был введён.

Можно смягчить настройкой шифрования; решение за вами.



4. Подготовка заранее и постоянный кэш

✘ Перепаковать веса в удобный для CPU порядок

Как если бы: разложить инструменты по ящикам заранее

В llama.cpp такая перепаковка Q4_K для x86 ускоряет чтение промпта на 61%, а генерацию не ускоряет (−2%) (PR #12332, замер автора на Ryzen; мной не воспроизводилось). У нас промпт упирается в диск, а генерация в память, поэтому польза не там, где болит. Загрузка при этом замедляется в разы.

Не для декодирования.

✘ Сохранить «горячий список» экспертов между запусками

Как если бы: заготовить к завтрашнему дню то, что готовили вчера

Закреплённый по другому тексту список закрывает лишь 40–59% обращений против 82–95% у обычного LRU. У разных задач разные любимцы, так что готовое «со вчера» не годится. Прогрев кэша простым чтением файла (22.8 ГБ за ≈ 7 с при 3.4 ГБ/с) полезен только при бюджете ≥ 22 ГБ и обходит лимит группы памяти; вопрос в вашем решении.

Список не нужен; прогрев — по желанию.

5. Ассемблер, встроенная графика и неиспользуемое железо

✘ Ассемблерные вставки для CPU и GPU

Как если бы: швейная машинка не шьёт быстрее, если ткань не успевают подавать

Видеокарта читает веса на ≈ 87% своего потолка (14.9 мс против 13.2). Ядра процессора на скалярном произведении выдают 18.6 ГБ/с на поток при необходимых 13.5, чтобы насытить память, то есть инструкции не узкое место. Разрыв CPU-пути (29 против 54 ГБ/с) в ожиданиях на барьерах и в простое между фазами, а не в выборе инструкций; ассемблер его не закроет.

Нужна перестройка работы потоков, не ассемблер.

✘ Встроенная графика Iris Xe для префила или инференса

Как если бы: ещё один повар на той же кухне с одной разделочной доской

Iris Xe берёт данные из той же оперативной памяти (LPDDR4X-4267, 128 бит, пик 68 ГБ/с; измерено 54), делит с процессором и корпус, и тепловой бюджет. Вычислительных ускорителей Intel (OpenCL, Level Zero) в системе нет, есть только Vulkan. Узкие места генерации и префила это память и диск, а не вычисления. Не проверено замером; вывод из измеренных потолков.

Скорее не поможет.

~ Невостребованные возможности процессора

Как если бы: в гараже есть мощный инструмент, которым не пользуются

В процессоре есть AVX-512 VNNI, VBMI2, BITALG и другие; K-квантовые скалярные произведения ggml используют только AVX2. Ядра VNNI, которые один раз распаковывают вес на 2–3 токена окна MTP, могут дать оценочно +10–20%. Не реализовано.

Единственное «железо в запасе».

~ Отложенные эксперты (метод KTransformers)

Как если бы: не ждать десерт, чтобы подать горячее: десерт принесут вместе со следующей переменой

Часть экспертов слоя досчитывается на CPU, пока GPU уже считает следующий слой, а их вклад добавляется на слой позже. Авторы метода (SOSP 2025, через пересказ в китайском источнике; мной не проверено) заявляют до ×1.45 при потере точности ≤ 0.5%. У нас GPU и CPU сейчас ходят по очереди (15 + 52 мс), поэтому это прямой путь сложить в «максимум». Меняет вывод модели.

Самая перспективная «приближённая» идея; нужно ваше решение.

✔ Процессор считает часть экспертов на префиле вместе с видеокартой

Как если бы: пока грузовики едут по узкой дороге, соседняя кухня готовит часть заказов сама

Шина PCIe 4.0 x4 (6.1 ГБ/с) давала потолок 167 ток/с, а процессор во время чтения запроса простаивал. Теперь последние 96 из 256 экспертов слоя считают ядра AVX-512 прямо из кэша памяти, остальные 160 едут на видеокарту. Замер: 142 → 202 ток/с на запросе в 1546 токенов, 22.1 → 15.2 с на 3099 токенах; токены совпадают с llama.cpp, ответы pi побайтно те же.

Сработало: единственная идея с «простаивающим железом», которая дала прирост на префиле.

6. Что говорят поиск и китайские источники

Поиск по китайским сайтам (知乎, CSDN, 掘金, bilibili и др.) почти не дал самих форумных постов с замерами на похожем железе: попадались пересказы и статьи. Практичные ориентиры нашлись такие: KTransformers (Университет Цинхуа) с отложенными экспертами и ядрами AMX (на нашем процессоре AMX нет); ik_llama.cpp с «слитыми» MoE-операциями и построчно переупорядоченными квантами (требует переквантования файла и не совмещается с гибридным режимом); тюнинг размещения экспертов (--n-cpu-moe), который на 12 ГБ видеокартах давал двукратный прирост, но для 4 ГБ почти не применим; и недавний кэш экспертов на GPU в llama.cpp, где для 4 ГБ мало места. Все числа из этих источников я не воспроизводил.

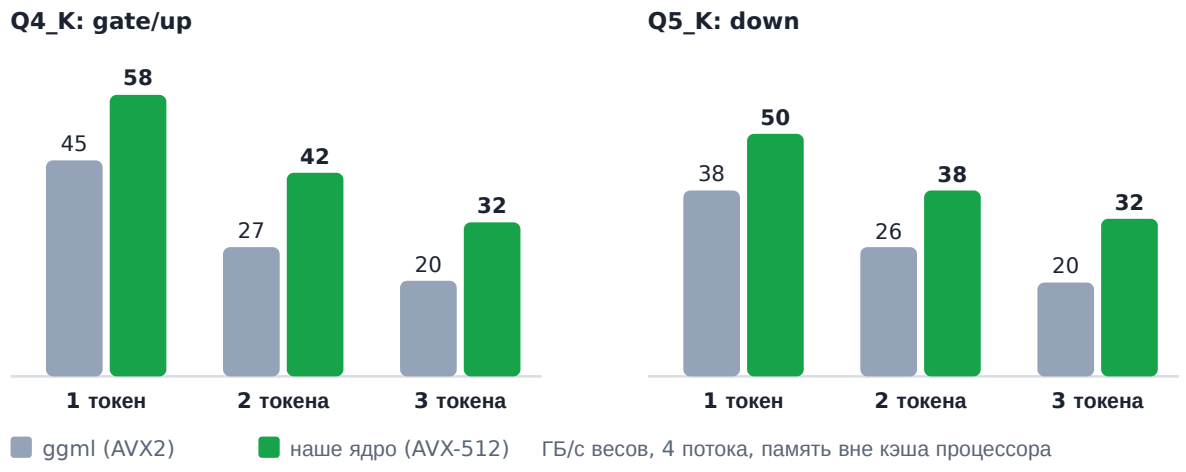
> Итог: на этом железе реально остались три пути: отложенные эксперты (приближённо), ядра AVX-512 и перестройка барьеров (точно), предсказание следующего слоя для малого объёма памяти (точно, но выигрыш ограничен диском). Остальное проверено и не окупается.

7 Что мы сделали для ускорения без изменения ответов

В разделе 5 мы нашли, что 52 из 75 мс шага уходят на процессор: он читает экспертов и считает по ним скалярные произведения. Мы сделали две вещи и обещали себе одно условие: ответы модели не должны измениться ни на один бит.

1. Новое ядро для AVX-512

> Как если бы: повар резал овощи на узкой доске, по 32 штуки за раз, и для каждого блюда чистил их заново. Теперь доска вдвое шире (64 штуки за взмах), а овощи чистятся один раз на все блюда, которые из них готовятся. «Блюда» это 2–3 токена, которые модель проверяет за один шаг; «чистка» это распаковка 4-битных весов.

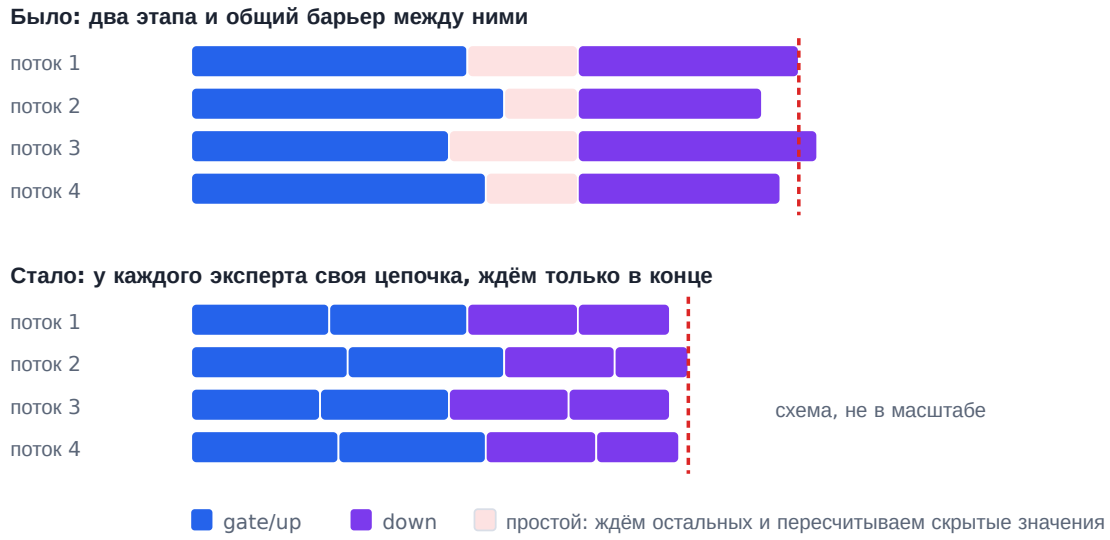


Ядро читает те же веса и даёт тот же результат бит в бит: мы сравнили его с эталонной функцией ggml на 200 случайных наборах (1–8 токенов) и получили ноль расхождений. Выигрыш растёт с числом

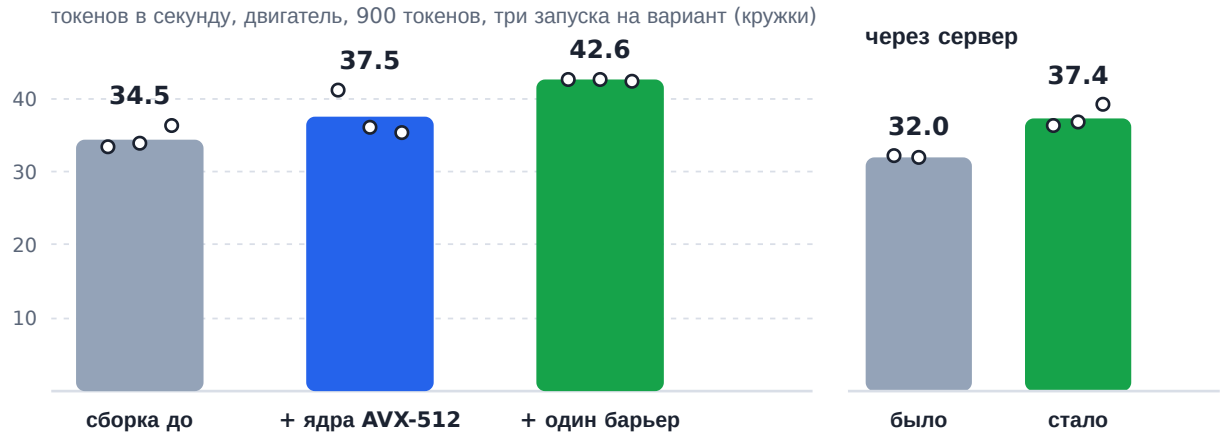
токенов в окне: на одном токене около +30%, на трёх около +60%.

2. Один барьер на слой вместо двух

Как если бы: на кухне все повара сначала режут, потом ждут друг друга у общей линии, и только потом все вместе начинают варить. Теперь у каждого блюда своя цепочка «нарезал, сразу варить»; ждать приходится лишь в конце, когда готово всё. Заодно исчезла последовательная работа одного повара между этапами (пересчёт скрытых значений).



Что это дало



Вариант	ток/с (900 токенов, три запуска)	к сборке «до»
сборка до (AVX2, два барьера)	33.3 · 33.9 · 36.3, в среднем 34.5	—
+ ядра AVX-512	41.2 · 36.1 · 35.3, в среднем 37.5	+8.8% (с шумом)
+ один барьер на слой	42.7 · 42.6 · 42.4, в среднем 42.6	**+23.4%**

Вариант	ток/с (900 токенов, три запуска)	к сборке «до»
через реальный сервер без лимита памяти	32.1 · 32.0 → 36.2 · 36.9 · 39.1	+17%

Раньше мы оценивали эти идеи в +10–20% и +5–10%. На деле ядра AVX-512 сами по себе дали около +9%, но с большим разбросом, а вместе с барьером получилось +23% и разброс почти пропал (42.4–42.7). Вероятная причина: на барьерах терялось больше времени, чем мы считали, и быстрые ядра как раз делали это ожидание заметнее.

проверка

Ответы не изменились

Генерация 600 токенов даёт одинаковую контрольную сумму в трёх режимах (старое, только AVX-512, всё вместе). Сверка с llama.cpp на итоговой сборке: 48 + 48 токенов и ещё 16 после длинного промпта совпали (fp16 и q8).

побитовое совпадение

оговорка

Что не перемерено

Все прогоны здесь при тёплом кэше и старте не горячее 55 °C. Профили с жёстким лимитом памяти (8, 12, 16, 22 ГБ) не пересняты: там часть времени уходит на диск, и выигрыш будет меньше. Профиль без лимита пересчитан через сервер.

8 Почему pi думал по 10–20 секунд и что мы с этим сделали

Простой вопрос «Сколько планет в солнечной системе?» ждал около 21 секунды, из них 18 — чтение запроса, а сам ответ занимал меньше трёх. Мы поймали запросы pi и нашли две причины.

причина 1

Одно и то же читалось заново

Каждый запуск pi отправляет одни и те же 1500 токенов (системный промпт и описание четырёх инструментов), и к ним дописывается вопрос. Запросы из разных сессий совпали побайтно, но движок каждый раз читал всё с нуля: 0 токенов переиспользовано. У этой модели есть «память-лента» (30 рекуррентных слоёв): её нельзя перемотать назад на середину, а запоминать нужный момент никто не умел.

читалось 1546 токенов из 1546

причина 2

Чтение шло в обход кэша памяти

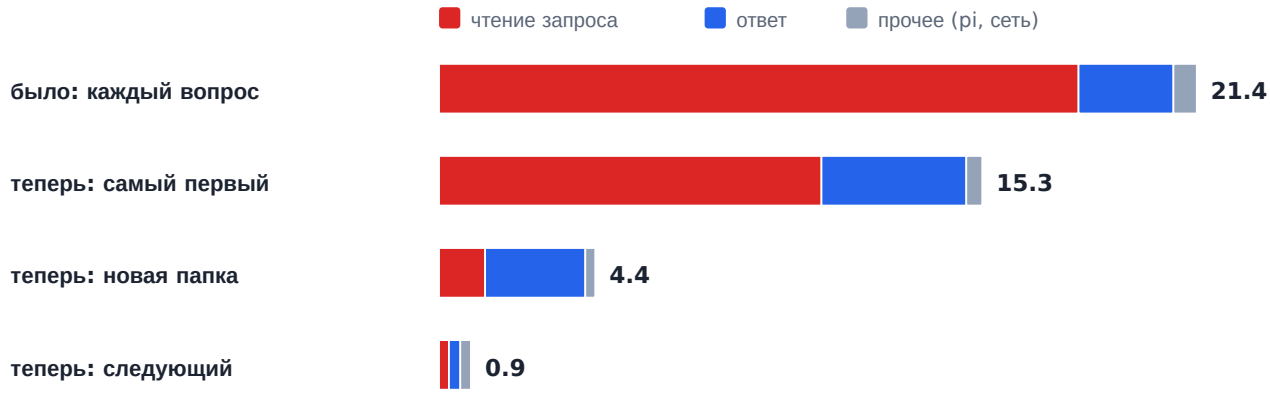
Чтобы прочитать запрос, движок прогоняет через видеокарту все 18.7 ГБ экспертов. Чтение с диска напрямую (O_DIRECT) не смотрит в кэш и работало на скорости SSD, 3.4 ГБ/с, даже когда вся модель лежала в оперативной памяти.

85.6 ток/с

Контрольная точка: закладка в книге

Как если бы: вместо того чтобы каждый раз перечитывать первые 1500 страниц, вы кладёте закладку на нужном месте. Движок теперь копирует состояние «ленты» и кэша внимания (84 МБ) в тот момент, когда закончилось общее начало разговора, и хранит копию в памяти и на диске (папка run/ckpt). Новая сессия pi находит закладку, возвращает состояние за миллисекунды и дочитывает только свой вопрос: около 20 токенов за 0.3 с вместо 18 с.

Рабочая папка записана в конце системного промпта pi, поэтому для каждой папки нужна своя закладка; мы ставим две: на самой границе и на 64 токена раньше. Во второй папке вторая закладка сохраняет всё, кроме последних строк, и первый вопрос занимает 4.4 с вместо 15–22. Закладки переживают перезапуск сервера. Побайтная проверка: свежий ответ, ответ из памяти и ответ с диска после перезапуска полностью совпали.



секунды от нажатия Enter до готового ответа; один и тот же вопрос в новой сессии pi

Чтение через кэш и стена шины

Как если бы: грузовики возят экспертов из склада в цех по узкой дороге. Раньше они ехали со склада на окраине (диск), теперь берут с ближайшей полки (кэш в памяти). Узкой осталась только сама дорога до видеокарты: PCIe 4.0 x4 даёт 6.1 ГБ/с, и это 3.07 с на один проход. Скорость выросла с 88 до 140 ток/с при потолке шины 167 ток/с, если всю работу делает видеокарта.



один проход чанка в 512 токенов гонит 18.7 ГБ экспертов через шину 6.1 ГБ/с: 3.07 с

Дополнительно найден и исправлен тонкий дефект: при чтении через кэш следующий чанк мог перезаписать буфер, из которого видеокарта ещё не закончила копировать. Теперь движок ждёт завершения копии. Потолок шины мы обошли другим способом (следующий абзац).

Процессор помогает видеокарте

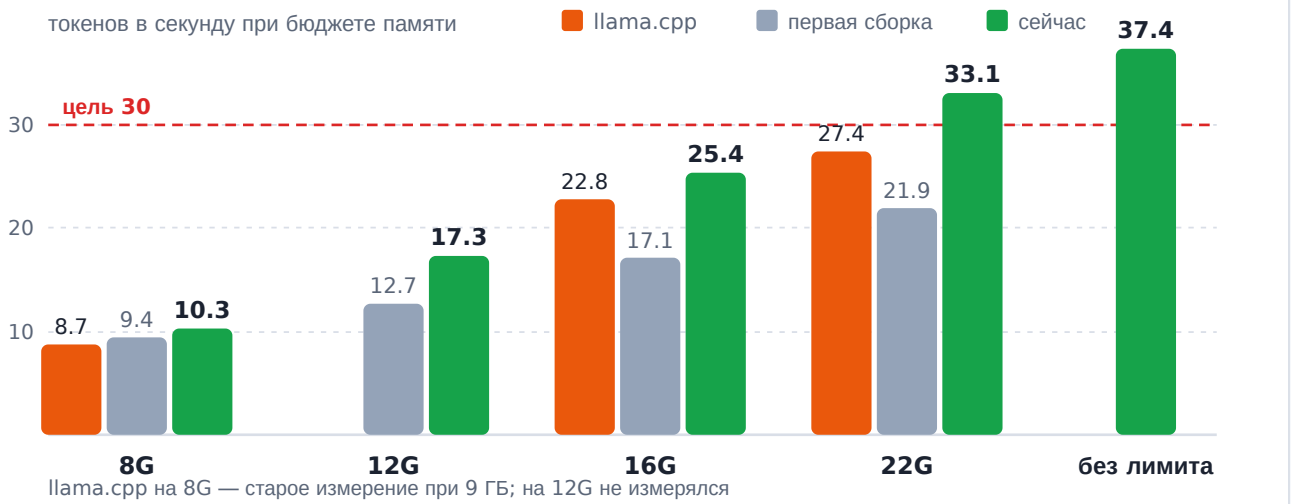
Как если бы: грузовики всё ещё едут по узкой дороге, но часть заказов цех не ждёт, а готовит на соседней кухне из продуктов, которые уже лежат рядом. Из 256 экспертов слоя видеокарта получает первые 160 по шине, а последние 96 в это же время считает процессор (ядра AVX-512 из раздела 7) прямо из файла модели в памяти; готовые строки копируются на видеокарту и складываются с её результатом. Шина везёт меньше, а процессор в это время раньше простаивал.

Экспертов на процессоре из 256	0	64	96	128	160
чтение запроса в 1546 токенов	10.85 s	8.66 s	**7.65 s**	7.80 s	8.88 s
токенов в секунду	142	178	**202**	198	174

Дальше доля процессора не помогает: он начинает считать дольше, чем едет шина. Запрос в 3099 токенов: 22.1 с было, 15.2 с стало (204 ток/с). Проверки: 16 токенов после длинного промпта совпали с llama.cpp, путь без этой опции не изменился (6 из 6 эталонных промптов совпали), ответ pi со свежим чтением, из памяти и с диска после перезапуска побайтно одинаков, 8 запросов pi без ошибок движка. Первый вопрос в сессии теперь читается 8.8 с при модели в кэше памяти и 10.8 с при холодном диске (было 11.5–18 с); процессор при этом греется сильнее, энергия на токен не измерялась.

9 Результаты

Скорость генерации почти целиком определяется тем, сколько экспертов лежит в оперативной памяти: чем больше бюджет, тем меньше обращений к SSD. Замер: жадная генерация с MTP, реальный код, 900 токенов, холодный старт и прогрев двумя запросами.



llama.cpp на 8G — старое измерение при 9 ГБ; на 12G не измерялся

Рядом с llama.cpp (та же машина, тот же бюджет, без MTP): при 16 ГБ 21–24 против 25.4, при 22 ГБ 27.4 против 33.1. То есть по генерации выигрыш умеренный — около 5–20%; основное преимущество

— длинный промпт (86 против 49 ток/с при 9 ГБ) и MTP на жадных запросах.



Проверка	Результат
Токены против llama.cpp (fp16 и q8 K/V)	совпадают: 48 + 48 токенов, ещё 16 после промпта в 3099 токенов
Черновики MTP против обычной генерации	вывод побайтно тот же (96 токенов)
Агент pi: вызовы инструментов, режим размышлений	работает; кэш префикса диалога переиспользуется
Сборка с нуля и тесты сервера	сборка проходит; 65 из 66 модулей тестов зелёные, один тест (подсчёт потоков сэмплера) иногда ловит гонку и падает примерно в одном запуске из пяти

9.1 А если взять ноутбук побольше или видеокарту для ПК (оценка)

Вопрос: сколько токенов в секунду выдали бы самые популярные локальные модели для кодинга на трёх топовых ноутбуках со 128–192 ГБ единой памяти и на четырёх видеокартах для ПК (флагманы серий 30, 40, 50 и RTX PRO) и во сколько раз это быстрее нашего ноутбука. Ничего не запускалось: взяты опубликованные характеристики, чужие замеры на том же железе и уже измеренные пределы нашей машины. Это оценка, а не замер: разброс зависит от платформы и модели, интервалы приведены в таблице и особенно широки при выгрузке в ОЗУ.

Важная оговорка про Strata. Наш форк работает только на Linux x86-64 с видеокартой NVIDIA и CUDA, поэтому ни один из трёх ноутбуков его не запустит: у MacBook Apple Silicon и Metal, у HP встроенная Radeon без CUDA, у ASUS процессор Arm и Windows на Arm. На настольных картах NVIDIA он запустится, но по README маршрутизируемые эксперты он не копирует в видеопамять, а считает на процессоре, так что большая память карты ему «как есть» не поможет. Цифры ниже — для обычных движков (llama.cpp, MLX, vLLM), где вся модель лежит в быстрой памяти, пока влезает. Что из нашего тюнинга переносится, показано отдельно.

Три ноутбука рядом с нашим

наш ноутбук

i7-11370H + RTX 3050 Ti

31 ГБ ОЗУ, видеокарта на 4 ГБ. Модель в быструю память не помещается: плотную часть держит видеокарта, эксперты считает процессор из ОЗУ, остальное подтягивается с SSD (3.5 ГБ/с).

172 ГБ/с видеопамять · 54 ГБ/с ОЗУ (замер)

Apple

MacBook Pro 16″, M5 Max

18 ядер CPU, 40 ядер GPU, 128 ГБ единой памяти. Одна память на CPU и GPU; движки MLX и llama.cpp (Metal). Под длительной нагрузкой 16″ держит GPU стабильно, около 70 Вт (Notebookcheck, по сниппету).

614 ГБ/с (Apple)

HP

ZBook Ultra G3a

Ryzen AI Max+ PRO 495: 16 ядер Zen 5, Radeon 8065S (40 CU), NPU 55 TOPS, до 192 ГБ памяти, из них до 160 ГБ как видеопамять. LPDDR5X-8000 на шине 256 бит для конфигураций 128 и 192 ГБ. CUDA нет: llama.cpp через Vulkan или ROCm. TDP до 100 Вт.

256 ГБ/с (расчёт), в замерах ≈ 215

ASUS

ProArt P16 (H7607), RTX Spark

NVIDIA N1X: 20 ядер Arm (10 Cortex-X925 и 10 Cortex-A725), GPU Blackwell на 6144 ядра CUDA, 128 ГБ LPDDR5X-9400 на шине 256 бит. 128 ГБ бывает только у полной 20-ядерной версии. Windows 11 на Arm, CUDA 13.4 её поддерживает; Linux не анонсирован. cTDP чипа 45–80 Вт.

≈ 300 ГБ/с (ServeTheHome)

Четыре видеокарты для ПК

Флагман по видеопамяти из каждой серии, официально представленный NVIDIA. Названия из первоначального списка уточнены: «RTX 4090 Ti» не выходила, у 3090 Ti и 4090 версий с большим объёмом нет, а RTX PRO 6000 Blackwell называется Workstation Edition. Сторонняя модификация RTX 4090 на 48 ГБ — не продукт NVIDIA, в таблицу я её не включала.

серия 30

GeForce RTX 3090 Ti

24 ГБ GDDR6X, шина 384 бит, 450 Вт. Флагман серии по памяти: версии с большим объёмом у NVIDIA нет (RTX 3090 тоже 24 ГБ).

≈ 1008 ГБ/с (сторонние данные)

серия 40

GeForce RTX 4090

24 ГБ GDDR6X, шина 384 бит. «RTX 4090 Ti» не выходила: по сообщениям 2023 года её отменили (источник — утечка, NVIDIA не подтверждала), признаков запуска в 2026 году я не нашла. Флагман серии по памяти — сама 4090.

≈ 1008 ГБ/с (по поисковой сводке; первоисточник не открылся)

серия 50

GeForce RTX 5090

32 ГБ GDDR7, шина 512 бит, 575 Вт, PCIe 5.0, 21 760 ядер CUDA (NVIDIA).

≈ 1792 ГБ/с (Notebookcheck)

RTX PRO

RTX PRO 6000 Blackwell

Официально «Workstation Edition»: варианта «Desktop Edition» нет. 96 ГБ GDDR7 с ECC, 600 Вт, PCIe 5.0. Версия Max-Q на 300 Вт, по странице NVIDIA, с той же пропускной способностью.

1792 ГБ/с (NVIDIA)



Скорость генерации упирается в пропускную способность памяти: на каждый токен нужно прочитать активные веса. Поэтому главная из характеристик — одна цифра, и у видеокарт она в разы выше, чем у ноутбуков. Но быстрой памяти у карты мало (24–96 ГБ), и модель, которая в неё не влезает, уходит частично в оперативную память ПК: тогда скорость определяет она. Цены не сравниваю: официальных цен для этих конфигураций я не нашла.

Самые популярные модели для кодинга

Модель	Тип	Параметры: всего / активных	Файл 4-бит, ГБ	За токен, ГБ	MTP	Популярность (скачивания)	Кодинг, самоотчёт
Qwen3.8-27B	плотная	27B / 27B	17.6	17.6	да	5.9 млн скачиваний GGUF за месяц; 5 млн pull в Ollama	LiveCodeBench v6 90.3 · SWE-bench Pro 61.7
Qwen3.6-35B-A3B	MoE	35B / 3B	22.4 (22.9 с MTP)	2.8	да	1.3 млн за месяц; Ollama qwen3.6: 7.3 млн pull (вместе с 27B)	SWE-bench Verified 73.4 · LiveCodeBench v6 80.4
Qwen3.6-27B	плотная	27B / 27B	17.6	17.6	да	372 тыс. за месяц	SWE-bench Verified 77.2 · LiveCodeBench v6 83.9
Qwen3-Coder-30B-A3B	MoE	30.5B / 3.3B	18.6	2.0	нет данных	Ollama qwen3-coder: 10.3 млн pull	—
gpt-oss-20b	MoE	21B / 3.6B	≈ 13.8	2.56	нет данных	546 тыс. за месяц; Ollama gpt-oss: 14.2 млн (вместе с 120b)	—
gpt-oss-120b	MoE	117B / 5.1B	63	3.5	нет данных	78 тыс. за месяц (плюс Ollama gpt-oss)	—
Qwen3-Coder-Next	MoE	80B / 3B	49.6	3.7	нет данных	Ollama: 2.1 млн pull; 148 тыс. за месяц	SWE-bench Verified 70.6

Модель	Тип	Параметры: всего / активных	Файл 4-бит, ГБ	За токен, ГБ	MTP	Популярность (скачивания)	Кодинг, самоотчёт
Flint	MoE	39.8B / 2.5B	26.0 (Q4_1), 22.9 (NVFP4)	1.6	нет данных	новая (21.10.2026); Ollama: 26 тыс. pull	SWE-bench Verified 74.2

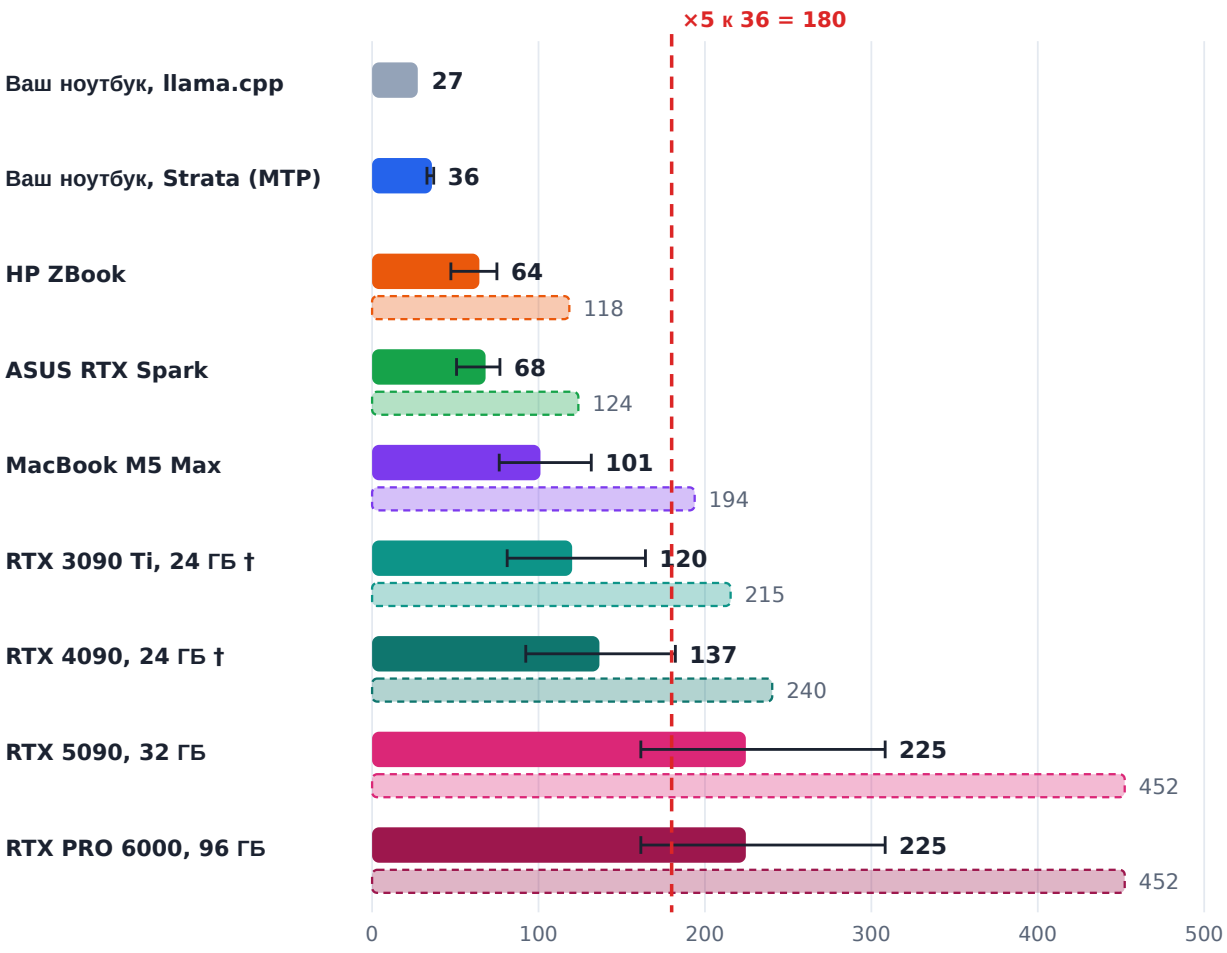
Популярность — по скачиваниям на Hugging Face и числу pull в Ollama на 26.10.2026. «За токен» — условный объём весов, который читается на один токен: у MoE это внимание и выбранные эксперты, у плотных вся модель. У Qwen3.6-35B-A3B взято из замера в разделе 11 (2.2 + 0.6 ГБ), у gpt-oss и Coder-30B посчитано по их config.json, у Qwen3-Coder-Next — по оценке раздела 11 для родственной Qwen3-Next-80B, у Flint — по размеру файла и доле активных параметров. Оценки качества — самоотчёт авторов, независимо их никто не проверял.

Сводная таблица: токенов в секунду

		Ноутбуки			Видеокарты для ПК (флагманы серий)			
Модель, ток/с	Ваш ноутбук	HP ZBook	ASUS RTX Spark	MacBook M5 Max	RTX 3090 Ti 24 ГБ	RTX 4090 24 ГБ	RTX 5090 32 ГБ	RTX PRO 6000 96 ГБ
Qwen3.8-27B плотная	**2.2** 1.8–2.8 оценка	**11.9** 8.9–13.2 ×5.4	**11.5** 8.6–12.7 ×5.2	**26** 21–30 ×11.9	**45** 36–51 ×20.4	**45** 36–51 ×20.5	**80** 63–90 ×36.1	**80** 63–90 ×36.1
Qwen3.6-35B-A3B MoE	**27** llama.cpp **36** Strata (замер)	**64** 47–75 ×2.4	**68** 51–77 ×2.5	**101** 76–132 ×3.7	**120**† 81–164 ×4.4	**137**† 92–182 ×5.0	**225** 162–308 ×8.2	**225** 162–308 ×8.2
Qwen3.6-27B плотная	**2.2** 1.8–2.8 оценка	**11.9** 8.9–13.2 ×5.4	**11.5** 8.6–12.7 ×5.2	**26** 21–30 ×11.9	**45** 36–51 ×20.4	**45** 36–51 ×20.5	**80** 63–90 ×36.1	**80** 63–90 ×36.1
Qwen3-Coder-30B-A3B MoE	**19.0** 16.0–23 оценка	**85** 62–100 ×4.5	**93** 69–106 ×4.9	**120** 89–161 ×6.3	**168** 120–232 ×8.8	**201** 146–270 ×10.6	**257** 182–365 ×13.5	**257** 182–365 ×13.5
gpt-oss-20b MoE	**16.0** 13.0–19.0 оценка	**69** 51–81 ×4.3	**74** 55–84 ×4.6	**106** 80–139 ×6.6	**150** 108–203 ×9.4	**176** 130–231 ×11.0	**233** 167–323 ×14.6	**233** 167–323 ×14.6
gpt-oss-120b MoE	**3.0** 2.0–4.0 оценка	**53** 39–61 ×17.8	**55** 41–62 ×18.4	**89** 68–113 ×29.7	**19.8**† 10.3–29 ×6.6	**20**† 10.4–30 ×6.7	**25**† 12.9–37 ×8.3	**202** 147–272 ×67.5
Qwen3-Coder-Next MoE	**7.0** 6.0–8.0 оценка	**51** 37–58 ×7.2	**52** 39–59 ×7.5	**86** 66–109 ×12.3	**38**† 21–55 ×5.5	**40**† 22–57 ×5.7	**55**† 30–80 ×7.8	**197** 143–262 ×28.1
Flint MoE	**30** 24–35 оценка	**101** 73–121 ×3.4	**114** 84–130 ×3.8	**132** 96–182 ×4.4	**109**† 66–159 ×3.6	**123**† 73–176 ×4.1	**285** 199–416 ×9.5	**285** 199–416 ×9.5

Цвет — по скорости, ток/с. «×N» — во сколько раз быстрее нашего ноутбука на той же модели (без MTP, у Qwen3.6-35B-A3B — относительно llama.cpp). Коэффициенты посчитаны до округления чисел в таблице. ≥ 150 | 60–150 | 25–60 | 10–25 | < 10

† — модель не помещается в видеопамять с запасом 3 ГБ на K/V-кеш и буферы, и часть экспертов лежит в ОЗУ процессора (допущение: 30 ГБ/с, разброс 15–45, хост неизвестен).



сплошная — без MTP (кроме измерения Strata); бледная пунктирная — потенциал с MTP, если движок его поддерживает
усы — разброс оценки; у нашего ноутбука значения измерены (Strata — уже с MTP), у остальных рассчитаны
† часть экспертов лежит в ОЗУ процессора (модель не влезает в видеопамять с запасом)

На модели, которую мы тюнили, выигрыш у обычного движка на нашем ноутбуке — ×2.4–3.7 у ноутбуков и ×4–8 у видеокарт (RTX 3090 Ti и 4090 держат её почти целиком в 24 ГБ, у остальных она влезает полностью). На плотных 27B, которые на нашем ноутбуке почти не ходят (около 2 ток/с, веса читает процессор), ноутбуки дают ×5–12, а видеокарты ×20–36: именно там большие машины открывают то, что на нашей недоступно. У RTX 5090 и PRO 6000 скорость на моделях, которые влезают в 32 ГБ, почти одинакова: при той же пропускной способности PRO 6000 даёт только объём, 96 ГБ позволяют держать целиком gpt-oss-120b и Qwen3-Coder-Next.

Что дал бы наш тюнинг

Приём из **Strata**	Зачем он нужен	Переносится на эти машины?
MTP: черновики и проверка 3 токенов за проход	Читает веса один раз на несколько токенов — выигрыш там, где всё упирается в память.	**Да.** На DGX Spark плотная Qwen3.6-27B (NVFP4, vLLM) дала 12.1 → 29.4 ток/с. Нужен движок с поддержкой MTP; для llama.cpp и MLX на этих платформах я это не проверяла.
K/V-кеш в 8 бит (q8)	Почти вдвое меньше памяти и чтения на длинном контексте.	**Да**, llama.cpp умеет квантовать K/V; для агентов с длинным контекстом это полезно, особенно на картах с 24–32 ГБ.
Контрольные точки начала диалога	Новая сессия агента не перечитывает системный промпт.	**Идея переносится**, реализация зависит от сервера.
Эксперты на CPU из кеша страниц, ядра AVX-512, потоковая подкачка	Нужны, когда модель не помещается в быструю память.	**Нет.** На 128–192 ГБ единой памяти и на карте с 96 ГБ вся модель лежит в быстрой памяти, и такой способ размещения и подкачки экспертов там не нужен.
Работа GPU и CPU по очереди на каждом слое	Способ разложить веса по двум памятям.	**Нет:** у ноутбуков здесь одна память на всех; у видеокарт, где модель влезает, всё считает GPU.
Наш форк «как есть» на настольной карте NVIDIA	Эксперты считает процессор из файла в памяти, плотная часть лежит на видеокарте.	**Запустится** (Linux x86-64, CUDA), но по README маршрутизируемые эксперты в видеопамять не копируются, так что скорость определит оперативная память ПК. По расчёту из замера шага в разделе 5 (52 мс при 29 ГБ/с) при 30–60 ГБ/с это порядка 50–90 ток/с с MTP: меньше, чем у обычного движка с экспертами в видеопамяти. Не проверялось.

С MTP, ток/с (потенциал)	HP ZBook	ASUS RTX Spark	MacBook M5 Max	RTX 3090 Ti 24 ГБ	RTX 4090 24 ГБ	RTX 5090 32 ГБ	RTX PRO 6000 96 ГБ
Qwen3.6-35B-A3B за шаг проверяются 3 токена, в среднем получается 2.7 (как в Strata)	**118** 87–137	**124** 92–139	**194** 148–249	**215** 140–296	**240** 156–324	**452** 326–617	**452** 326–617
Qwen3.6-27B, Qwen3.8-27B ×2.4 к обычной скорости (замер на DGX Spark); разброс — множитель 2.0–2.6	**29** 17.7–34	**28** 17.2–33	**63** 43–78	**109** 72–132	**109** 72–132	**192** 127–233	**192** 127–233

×5 к нашим 36 ток/с — это 180 ток/с. Без MTP его по оценке достигают RTX 5090 и PRO 6000 на Qwen3.6-35B-A3B (≈ 225); из ноутбуков — только M5 Max и только с MTP (≈ 194). Это потенциал, а не результат: для MTP нужен движок с его поддержкой, а на Metal у нашего форка его нет.

Чтение промпта: что известно

Платформа	Модель и движок	Чтение промпта, ток/с
Ваш ноутбук	Qwen3.6-35B-A3B, Strata	84–123 при бюджете 9 ГБ, около 200 при 22 ГБ; llama.cpp 49 (замер)
Strix Halo (395)	Qwen3-30B-A3B Q4_K_M, Vulkan	1142 (visorcraft)
Strix Halo (395)	Qwen3.6-35B-A3B Q8_0, Vulkan / ROCm	1047 / 1014 (level1techs)
DGX Spark	Qwen3-Coder-30B-A3B Q8_0, llama.cpp	2933 (jetsonhacks)
DGX Spark	gpt-oss-20b / gpt-oss-120b, llama.cpp	3611 / 1689 (jetsonhacks)
MacBook M5 Max	—	надёжных замеров не нашла (один блог заявляет около 1.9 тыс., но методика не раскрыта)
ASUS RTX Spark	—	замеров нет
RTX 3090	Qwen3 32B Q4_K_M, llama.cpp	1304 (computingforgeeks)
RTX 4090	Qwen3-30B-A3B Q4_K / gpt-oss-20b MXFP4, 4K	6159 / 8369 (Hardware Corner)
RTX 5090	Qwen3 32B Q4_K_M / Qwen3-30B-A3B Q4_K_XL	3657 / 6630 (Hardware Corner)
RTX PRO 6000	gpt-oss-120b MXFP4 / Q8_0, 4K	4579 / 6513 (Hardware Corner)

Чтение промпта упирается не в память, а в вычисления (здесь важны TOPS), и для агента с длинным контекстом оно не менее важно, чем генерация. Цифр для M5 Max и RTX Spark я не нашла, поэтому здесь оценки нет. Для сверки по генерации: вендор Flint заявляет более 150 ток/с на RTX Spark с 64 ГБ (свой движок, короткий контекст; пересказ), а наша формула даёт для Flint на RTX Spark около 114 (84–130): заявление вендора выше расчётного разброса.

Как это посчитано

Ноутбуки: `ток/с = 1000 ÷ (ГБ за токен ÷ эффективная ГБ/с × 1000 + с)`, где с — постоянные накладные расходы на токен в миллисекундах (запуск ядер, маршрутизация экспертов). Видеокарты: то же, но

эффективная пропускная способность — 80% паспортной, с — 0.3 мс для плотных и 2.5 мс для MoE; если модель не влезает в видеопамять с запасом 3 ГБ, недостающая доля экспертов (у плотных — слоёв) читается процессором из ОЗУ на 30 ГБ/с. Параметры подобраны по чужим замерам на том же или близком железе (для некоторых машин — на других моделях). Разброс: пропускная способность от ×0.75 (для Mac ×0.85, для видеокарт ×0.8) до ×1.10 (у видеокарт ×1.12), с от ×0.6 до ×1.5, скорость ОЗУ-хоста от ×0.5 до ×1.5. Для RTX 3090 Ti накладные расходы с умножены на 1.4: в замерах Arcee (вендор, docs.arcee.ai) на малых моделях RTX 4090 быстрее RTX 3090 на 28–31% при +8% пропускной способности; поэтому на плотных моделях колонки 3090 Ti и 4090 почти совпадают (разница в пределах округления), а на MoE 3090 Ti медленнее.

Платформа	Паспорт, ГБ/с	Эффективная, ГБ/с	c, мс	На чём откалибровано
HP ZBook (Strix Halo)	256	215	2.5	Замеры на Ryzen AI Max+ 395 (та же 256-битная память), не на PRO 495: Qwen3-30B-A3B 86, gpt-oss-120b 36–53, Qwen3.6-27B 11.8 (visorcraft, level1techs); максимум чтения ≈ 215 ГБ/с — со слов level1techs, из вторых рук.
ASUS RTX Spark	300	205	1.0	Самого RTX Spark замеров нет. Опора — DGX Spark (GB10, 273 ГБ/с): gpt-oss-120b 52.9, gpt-oss-20b 79.7, Qwen3-Coder-30B Q8_0 60.0, Qwen3.6-27B NVFP4 12.1 (jetsonhacks, classmethod); пересчёт на +10% пропускной способности. Windows на Arm — риск по софту, нижняя граница шире.
MacBook M5 Max	614	520	4.5	Qwen3.6-35B-A3B на MLX около 118–126 (сообщество, llmcheck); Qwen3-30B-A3B на M4 Max 109.7 (arXiv 2601.19139); 70B плотная ≈ 13 (llmcheck, без названного источника). У MoE на Apple доля пропускной способности заметно ниже, чем у плотных: сказываются накладные расходы на токен (параметр c).
RTX 3090 Ti / 4090 / 5090 / PRO 6000	1008 / 1008 / 1792 / 1792	80% паспортной	0.3 плотные, 2.5 MoE	Модель целиком в видеопамяти: RTX 3090 — Llama 3.1 8B 147, Qwen3 32B 39; RTX 5090 — Llama 8B 278, Qwen3 32B 61–75, Qwen3-30B-A3B 234, Llama 70B IQ3_XXS 53; RTX 4090 — Qwen3-30B-A3B 207, gpt-oss-20b 191; PRO 6000 — gpt-oss-120b 210–221, Llama 70B 32 (computingforgeeks, 5090, 4090, PRO 6000). Для 3090 Ti пропускная способность взята такой же, как у 4090.

Платформа	Паспорт, ГБ/с	Эффективная, ГБ/с	c, мс	На чём откалибровано
ПК-хост для экспертов в ОЗУ	—	30 (15–45)	—	Допущение, а не данные: gpt-oss-120b на RTX 3090 с выгрузкой экспертов дал около 30 и около 15 ток/с (два неформальных замера), а ещё две точки — 1.8 (3090) и 9.6 (5090) — расходятся с ними в разы, и хост в них описан неполно (Hardware Corner).
Наш ноутбук	172 / 54	153 / 29	—	Метод раздела 11: чтение плотных весов видеокартой (153 ГБ/с) плюс чтение экспертов процессором (29 ГБ/с); на нашей модели он даёт 27.9 ток/с без MTP при замере 28.5. Для llama.cpp коэффициент 0.83.

Сверка с теми же замерами, по которым подбирались параметры (это не независимая проверка): из 25 контрольных точек в 20 измеренное значение или диапазон пересекается с расчётным разбросом (критерий строгий, без допуска). Промахи (5) оставлены в таблице и помечены красным.

Платформа	Что измерено	Расчёт, ток/с	Измерено, ток/с	Пересекает разброс?
HP ZBook (395)	Qwen3-30B-A3B Q4_K_M	85 (62–100)	86	да
HP ZBook (395)	gpt-oss-120b	53 (39–61)	36–53	да
HP ZBook (395)	gpt-oss-20b	69 (51–81)	49–51	ниже расчёта
HP ZBook (395)	Qwen3.6-27B Q4_K_M	12.4 (9.3–13.8)	11.8	да
DGX Spark	gpt-oss-120b	50 (38–56)	53	да
DGX Spark	gpt-oss-20b	68 (50–76)	80	выше расчёта
DGX Spark	Qwen3-Coder-30B-A3B Q8_0	55 (41–61)	60	да
DGX Spark	Qwen3.6-27B NVFP4	12.1 (9.0–13.3)	12.1	да
M5 Max	Qwen3.6-35B-A3B MLX 4-bit	101 (76–132)	118–126	да
M5 Max	Llama 3.3 70B 4-bit	12.4 (10.4–13.9)	13.0	да
M4 Max	Qwen3-30B-A3B 4-bit	123 (90–166)	110	да
RTX 3090	Llama 3.1 8B Q4_K_M	146 (116–166)	147	да
RTX 3090	Qwen3 32B Q4_K_M	37 (30–42)	39	да
RTX 5090	Llama 3.1 8B Q4_K_M	269 (212–309)	278	да
RTX 5090	Qwen3 32B Q4_K_M / Q4_K_XL	71 (56–80)	61–74	да
RTX 5090	Llama 3.3 70B IQ3_XXS	52 (42–59)	53	да
RTX 4090	Qwen3-30B-A3B Q4_K	201 (146–269)	207	да

Платформа	Что измерено	Расчёт, ток/с	Измерено, ток/с	Пересекает разброс?
RTX 4090	gpt-oss-20b MXFP4	176 (130–231)	191	да
RTX 5090	Qwen3-30B-A3B Q4_K_XL	257 (182–364)	234	да
RTX PRO 6000	gpt-oss-120b MXFP4 / Q8_0	202 (147–272)	210–221	да
RTX PRO 6000	Llama 3.3 70B Q4_K	33 (27–38)	32	да
RTX 3090 + ОЗУ	gpt-oss-120b, эксперты в ОЗУ (замер ≈ 30, неформальный)	19.7 (10.3–29)	30	выше расчёта
RTX 3090 + ОЗУ	gpt-oss-120b, эксперты в ОЗУ (замер ≈ 15, ОЗУ DDR4-3200)	19.7 (10.3–29)	15.0	да
RTX 3090 + ОЗУ	gpt-oss-120b, эксперты в ОЗУ (хост описан неполно)	19.7 (10.3–29)	1.8	ниже расчёта
RTX 5090 + ОЗУ	gpt-oss-120b, эксперты в ОЗУ (хост не указан)	25 (12.9–37)	9.6	ниже расчёта

Оговорки

Что	Как это влияет
Замеров на самих машинах нет	Цифры для сравниваемых машин — расчёт по чужим замерам на близком железе; разброс зависит от платформы и модели (интервалы приведены в таблице, при выгрузке в ОЗУ они особенно широки). У нашего ноутбука значения измерены или посчитаны по методу раздела 11.
Названия видеокарт в запросе	«RTX 4090 Ti» не выходила (по сообщениям 2023 года отменена, NVIDIA не подтверждала), версий 3090 Ti и 4090 с большим объёмом памяти у NVIDIA нет, «RTX PRO 6000 Blackwell Desktop Edition» официально называется Workstation Edition. В таблице — флагманы серий: 3090 Ti, 4090, 5090 и PRO 6000.
Видеокарты: выгрузка в ОЗУ (†)	Для 24 и 32 ГБ gpt-oss-120b и Qwen3-Coder-Next не влезают, и скорость зависит от ПК, которого я не знаю. Замеры gpt-oss-120b с выгрузкой расходятся от 1.8 до 30 ток/с; в расчёте взято 30 ГБ/с для ОЗУ, а разброс 15–45 даёт широкие границы.
ZBook: замеры на Ryzen AI Max+ 395, а в ноутбуке 495	Память той же 256-битной шины; у HP для конфигураций 128 и 192 ГБ заявлено LPDDR5X-8000, другие источники пишут 8533 (273 ГБ/с).
RTX Spark: Windows на Arm, замеров нет	CUDA 13.4 поддерживает Windows на Arm, но результатов llama.cpp на этих ноутбуках я не нашла; нижняя граница сделана шире.
Mac: параметры MoE подобраны по слабым источникам	Опора — сообщество и один препринт; один блог с 71–78 ток/с я отбросила: версии софта в нём не сходятся с датой, методика не раскрыта.
Длительная нагрузка и питание	ZBook до 100 Вт, RTX Spark 45–80 Вт (cTDP чипа), M5 Max около 70 Вт устойчиво, у видеокарт 450–600 Вт (3090 Ti 450, 5090 575, PRO 6000 600); троттлинг при долгой генерации не измерен.
Короткий контекст	Расчёт не учитывает чтение K/V-кеша; на длинном контексте скорость ниже: в замерах Qwen3-30B-A3B на RTX 5090 дал 234 ток/с на 4K и 111 на 32K, gpt-oss-120b на PRO 6000 — 210 на 4K и 100 на 128K.

10 Длинный контекст и видеопамять: окно 64K

Вы спросили про 128K, а потом попросили хотя бы 64K на видеокарте. Итог: 128K на карте с 4 ГиБ не помещается, а 64K теперь работает и стоит по умолчанию. На токен окна приходится около 13 КБ (K/V-кэш в q8 11 968 байт: 11 слоёв внимания × 2 головы × 256 × 2 для K и V, и ещё около 1 КБ буферов). Окно 64K до правок падало с «out of memory» при запуске, 128K потребовало бы 1.46 ГиБ одного K/V при весах 2.03 ГиБ. Мы нашли в видеопамяти лишнее и сжали его без изменения вывода модели; заодно заполненный контекст перестал замедлять декод так сильно.

Что замедляло длинный контекст

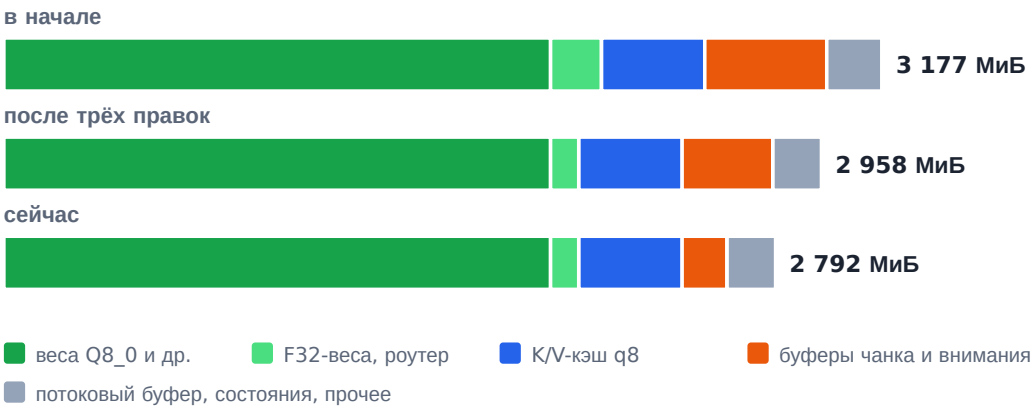
Как если бы: восемь читателей одного читального зала по очереди берут с полки одну и ту же книгу, вместо того чтобы прочитать её вместе. В модели 16 запросных голов делят 2 K/V-головы (по 8 на каждую), а старое ядро внимания читало кэш для каждой запросной головы отдельно, и при MTP ещё для каждой из трёх проверяемых позиций. Новое ядро читает чанк кэша один раз на все 8 голов и считает их вместе. Арифметика и порядок суммирования те же, поэтому результат побитово тот же: сравнение старого и нового ядра на реальных промптах (1.5K и 14K токенов, MTP 0 и 2) дало 0 отличий из 27–275 млн значений.

Замер (остывший процессор, окно 32K)	Было	Стало
время ядра внимания на вызов, 1.5K, MTP 2	246 мкс	56 мкс
то же при заполнении 14K, один токен	931 мкс	≈ 250 мкс или меньше
декод без MTP: 1.5K / 14K / 28K, мс на токен	35.1 / 47.6 / 55.0	32.9 / 36.5 / 38.8
шаг с MTP 2 при 28K, мс	141.6 и 147.5	84.8
шаг с MTP 2 при 1.5K, медианы шести запросов, мс	69.8 · 75.6 · 76.3	64.5 · 65.8 · 83.0

Плата за заполненное окно на шаге с MTP упала с ≈ 70 мс (почти вдвое) до ≈ 15–20 мс. На коротком контексте скорость та же в пределах шума (±15%: тепловое состояние процессора важнее, чем эффект). Первые замеры в этой серии вышли хуже старого ядра из-за помех, их пришлось перепроверить прогонами ABBA. Неточности: на 14K в серии без MTP набралось всего 11 токенов (модель остановилась), шаг при 28K измерен одним запуском. На окне 64K после промпта в 40 891 токен: чтение заняло 362 с (113 ток/с в среднем, внимание на длинном контексте дороже), декод 37.7 мс на токен без MTP и 102.5 мс на шаг с MTP.

Куда уходят 4 ГиБ и что мы убрали

Как если бы: чемодан, в который вы складывали вещи не глядя. Мы подменили вызов выделения памяти cudaMalloc своей обёрткой, которая записывает каждый запрос, и разложили 985 выделений по местам в коде. Вот итог при окне 32K на трёх этапах.



всё, что движок выделил через cudaMalloc при окне 32 768 токенов;
вне этого ещё контекст CUDA, cuBLAS и коды ядер (≈ 0.6–0.8 ГиБ)

Находка	Что было	Сэкономлено (32K)
Буфер частичных сумм внимания	рассчитан на 1024 строки запросов (16 голов × 64 токена), а ядру декода нужно не больше 128 (16 голов × 8 токенов); чанки промпта идут через GEMM и его не используют	112.9 МиБ
Дубль весов роутера	копия роутера в F32 и вторая копия с добавленной строкой гейта общего эксперта; первые 256 строк у них идентичны	82.0 МиБ
Буфер результатов CPU-пути	выделен на весь чанк (4096 записей), а CPU-путь слоя берёт не больше 128 строк	24.0 МиБ
Внимание промпта по одной K/V-голове	буферы K/V в fp16 и оценок внимания держали обе K/V-головы сразу; теперь одну, те же GEMM и то же ядро softmax	40.8 МиБ (на 64K: 57.5)
Общая арена для буферов трёх фаз слоя	буферы миксера GDN, миксера внимания и блока MoE никогда не живут одновременно, но занимали разную память; теперь одну область размером с самую большую фазу	126 МиБ (78 вместо 204; на 64K: 110 вместо 236)
Итого		**≈ 385 МиБ**

Ни одна правка не меняет арифметику. Свободно после инициализации (окно → стало):

Окно	в начале	после трёх правок	сейчас
32 768	0.14 ГиБ	0.33 ГиБ	0.49 ГиБ
49 152	—	0.13 ГиБ	—

Окно	в начале	после трёх правок	сейчас
57 344	—	0.08 ГиБ: падает после инициализации	0.21 ГиБ
65 536	out of memory при запуске	out of memory при запуске	**0.12 ГиБ — работает**
69 391	—	—	0.10 ГиБ: падает после инициализации; 0.11 ГиБ с --att-scores 4 и тайловым вниманием — работает (потолок)

0.08 ГиБ не хватает (движок падает на загрузке первых модулей ядер), 0.12 ГиБ хватает, 0.10 ГиБ нет: практический минимум около 0.11 ГиБ. Буферы K/V выделяются шагами по 2 МиБ, поэтому любое окно между 65 536 и 69 391 сразу стоит +44 МиБ и лишь теряет память: потолок этой карты 69 391 токен в q8 (следующая ступень 73 246 нужна ещё 44 МиБ). Проверки итоговой сборки: контрольные суммы 96 токенов при заполнении 1.5K и 14K с MTP 0 и 2 равны побитово проверенным; сверка токенов с llama.cpp 6 из 6; 12 форм промптов (от 5 до 1100 токенов, чекпойнты включены) проходят на окне 65 536; промпт в 40 891 токен на окне 65 536 даёт одни и те же 64 токена с MTP 0 и MTP 2. Один пробел не закрыт: пик по nvidia-smi упал меньше, чем выросла свободная память по счётчику движка; вероятно, драйвер занимает часть освободившегося после инициализации.

Внимание промпта по тайлам ключей

Как если бы: вместо того чтобы для каждой небольшой порции вопросов заново переписывать начисто всю библиотеку, вы листаете её тайлами: каждый том переписывается один раз, и сразу на все вопросы порции. До 4096 ключей промпт считает внимание как раньше, строка целиком, побитово тем же результатом. Дальше внимание идёт тайлами ключей (1024 при --att-scores 8, 512 при 4): тайл K переводится в fp16 один раз, одно GEMM даёт оценки для всех вопросов порции, ядро с бегущим softmax берёт максимум тайла, пересчитывает уже накопленные строки вывода и прибавляет сумму тайла, тайл V переводится в fp16, второе GEMM прибавляет веса × V к выводу, в конце строки делятся на накопленную сумму. Раньше для каждой порции из 8–128 вопросов заново переводились в fp16 весь K и весь V, из-за этого длинные промпты тормозили; там же держалась копия всего контекста в fp16 (33.5 МиБ на 64K), теперь только тайл (2 МиБ).

Измерение	Было	Стало
чтение промпта в 14K токенов (ABBA), с	81.3 · 73.2	71.4 · 70.5
чтение промпта в 40 891 токен на окне 64K, с	362 · 348 (113 ток/с)	278 (147 ток/с)
арена буферов при окне 64K, МиБ	110	80
свободно после инициализации при окне 64K	0.12 ГиБ	0.14 ГиБ (≈ 0.15 с --att-scores 4)

Измерение	Было	Стало
наибольшее окно, которое запускается	65 536	**69 391** (с --att-scores 4, 0.11 ГиБ в запасе)

Округление отличается от строки целиком (веса округляются до деления на сумму), поэтому выше 4096 ключей вывод не побитовый. Оба пути на одних и тех же входах, все слои и порции: на 47 млн значений промпта в 1.5K (тайлы принудительно с 512 ключей) расхождение не больше 4.3e-03 при значениях до 15.6, 1.4e-04 от среднеквадратичного значения; на заполнении 14K (404 млн значений) не больше 6.2e-03 и 1.0e-04. Сверка токенов с llama.cpp 6 из 6 при принудительных тайлах для каждой порции с больше чем одним ключом. На синтетическом промпте в 1.5K 96 токенов с тайлами расходятся с третьего токена (почти равные логиты; контрольная сумма 090e5f7a2590 против c33f0af2b4c9), после промпта в 41K 64 токена тоже отличаются (8d3eb7e99100 против 5c9423fabb62). Оба пути округляют веса до fp16, расхождение между ними на уровне такого округления. Промпты pi длиной до 4096 токенов идут прежним путём и вывод не меняют.

Что ещё можно ужать

Идея	Выигрыш	Цена и риск
Плотные веса Q8_0 → Q6_K или Q5_K	≈ 0.45 / 0.68 ГиБ (расчёт)	меняет вывод модели (повторное квантование); зато немного быстрее. Нужно ваше решение
K/V-кэш q8 → q4	≈ 0.35 ГиБ на 64K (0.73 на 128K)	меняет вывод; потеря точности внимания на длинном контексте
Окно K/V только у блока MTP (черновики)	≈ 68 МиБ на 64K	вывод не меняется (проверка всё равно делает основная модель), падает только доля принятых черновиков на длинном контексте
Группа MMQ 16 → 8 (потоковый буфер 70.5 → ≈ 35 МиБ)	≈ 35 МиБ	не измерено: больше мелких копий и запусков MMQ
Меньшие рабочие области cuBLAS, лимиты стека и кучи	неизвестно (контекст, cuBLAS и коды ядер занимают 0.6–0.8 ГиБ)	может изменить выбор алгоритма cuBLAS и вместе с ним округление
Выходная матрица (515 МиБ, Q8_0) на CPU	515 МиБ	читается на каждом токене: на CPU это ≈ 8–10 мс к шагу. Не рекомендуется
Старая часть K/V в оперативной памяти, внимание к ней считает CPU	окно 128K	большая работа; на каждом шаге ≈ +0.22 мс за каждую 1K токенов вне VRAM (расчёт по 54 ГБ/с). Решение за вами

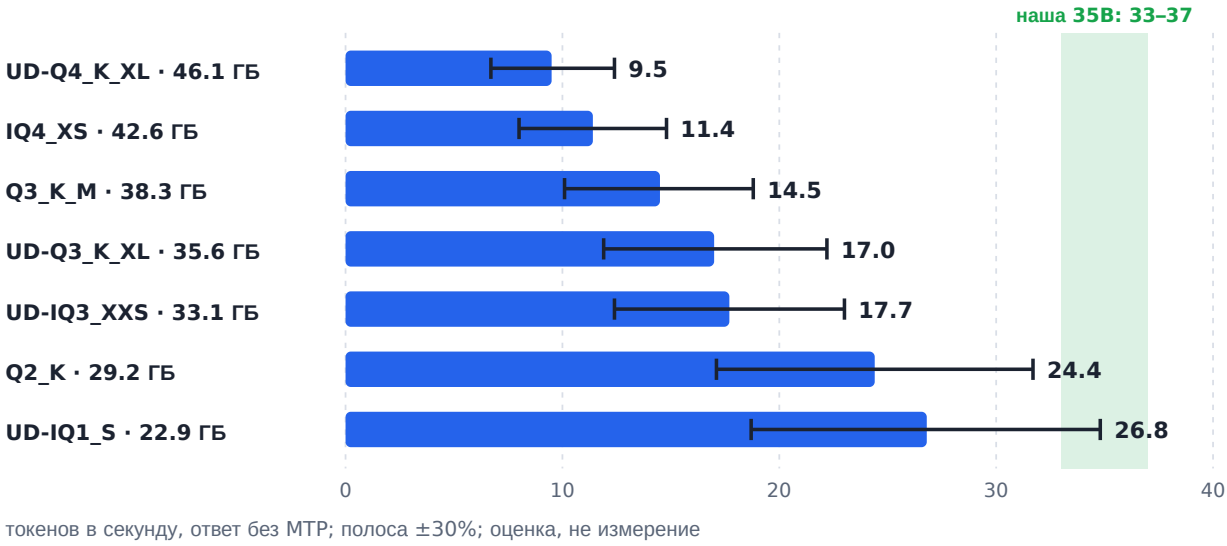
Вывод: 64K работает (≈ 0.15 ГиБ в запасе), наибольшее окно 69 391 токен. Окно K/V у MTP и меньшая группа MMQ дали бы вместе ещё около 8K токенов (68 + 35 МиБ при 13 КБ на токен), но шагами по 44 МиБ. 128K на карте с 4 ГиБ возможно только двумя способами: потерять точность (q4 для K/V и более лёгкие веса) или вынести старую часть кэша в оперативную память и считать внимание к ней на процессоре.

11 А если взять модель побольше: Qwen3-Next-80B-A3B (оценка)

Вопрос: сколько токенов в секунду выдавала бы на этом ноутбуке модель Qwen3-Next-80B-A3B (Qwen/Qwen3-Next-80B-A3B-Instruct на Hugging Face). Ничего не скачивалось и не запускалось: взяты метаданные модели (config.json и размеры файлов квантований) и уже измеренные пределы этой машины. Это оценка, а не замер.

	Qwen3.6-35B-A3B (наша)	Qwen3-Next-80B-A3B
слоёв (с полным вниманием / линейных)	40 (10 / 30)	48 (12 / 36)
экспертов, из них на токен	256, 8	512, 10
вид эксперта	2048 × 512 × 3	2048 × 512 × 3
все эксперты, ГБ (UD-Q4_K_XL)	≈ 20	≈ 43
экспертов читается на токен, ГБ	≈ 0.6	≈ 0.85
плотные веса на видеокарте, ГБ	≈ 2.2	≈ 2.6
файл UD-Q4_K_XL, ГБ	22.9	46.1

Как если бы: тот же повар на той же кухне, но кладовая вдвое больше, а в оперативную память помещается только её половина. На каждое слово нужно по 10 «книг» из 512 на слой: те, что лежат в памяти, процессор берёт сразу, остальные дочитываются с SSD, и именно эта часть решает скорость. Расчёт: время на токен = чтение плотных весов видеокартой (153 ГБ/с) + чтение экспертов процессором (29 ГБ/с) + штраф за экспертов, не вошедших в кэш; штраф взят из измеренной кривой нашей модели (скорость при кэше 8, 12, 16 и 22 ГБ), умноженной на отношение трафика (1.5×). Проверка метода: для нашей модели он даёт 27.9 ток/с без MTP при полном кэше, измерено 28.5.



токенов в секунду, ответ без MTP; полоса ±30%; оценка, не измерение

Версия	Файл	Экспертов в кэше	Наш движок, ток/с	llama.cpp, ток/с
UD-Q4_K_XL	46.1 ГБ	57%	**≈ 9.5** (6.7–12.4)	≈ 8
IQ4_XS	42.6 ГБ	62%	≈ 11 (8–15)	≈ 9
Q3_K_M	38.3 ГБ	69%	≈ 14.5 (10–19)	≈ 12
UD-Q3_K_XL	35.6 ГБ	75%	≈ 17 (12–22)	≈ 14
UD-IQ3_XXS	33.1 ГБ	81%	≈ 17.7 (12–23)	≈ 15
Q2_K / UD-Q2_K_XL	29.2 / 30.1 ГБ	90–93%	≈ 23–24 (16–32)	≈ 19–20
UD-IQ1_S / IQ1_M	22.9 / 24.4 ГБ	все	≈ 25–27 (18–35)	≈ 21–22

Для сравнения, наша Qwen3.6-35B-A3B даёт 33–37 ток/с. Значит, 80B в приличном качестве (квантования от Q3 и выше) в 2–4 раза медленнее, а 25 ток/с и больше возможны только на 1–2-битных версиях, где страдает качество. Оценка построена для плана «плотное на видеокарте, эксперты на процессоре из кэша страниц»; в столбце llama.cpp взят коэффициент 0.83 из прежних сравнений на нашей модели.

Оговорки

Что	Как это влияет
Наш движок эту модель не запустит	загрузчик принимает только архитектуру qwen35moe. Ограничения по геометрии (голова 256, состояние 128, свёртка 4, до 512 экспертов, top-10) подходят, так что адаптация вероятна, но не проверена. llama.cpp знает архитектуру qwen3next (есть в master) и запустит сейчас
MTP	в config.json нет ключей MTP-головы, так что в расчёте её нет; будь она и поддерживай её движок, добавилось бы +20–35%

Что	Как это влияет
Окно контекста	плотные веса на карте на 0.34 ГиБ больше, чем у нашей модели: окно 64K не поместится, 32K на грани (по расчёту)
Чтение промпта	за проход через эксперты читается 46 ГБ вместо 18.7: грубо 50–70 ток/с, то есть первый запрос pi в 2K токенов около 30–40 с вместо 9 с, а 30K токенов около 8–10 минут
Допущения	локальность маршрутизации у 512 экспертов такая же, как у наших 256 (не проверено), кэш страниц около 23 ГБ при свободной машине, разброс ±30%

12 Честно: что осталось

Мало памяти — медленно при жёстком лимите 8 ГБ около 10 ток/с: эксперты читаются с SSD. 30 ток/с достигаются, когда модель целиком в кэше (≈ 23 ГБ); профиль по умолчанию auto не ставит лимит, и ядро само отдаёт кэш другим программам. 60 ток/с на этом железе недостижимы (раздел 5).

Тепловой потолок на длинной генерации процессор упирается в 100 °C; выигрыш дальше — только в меньшем числе операций на токен.

Идеи на потом отложенные эксперты (приближённо), предзагрузка для малого объёма памяти, более плотное сжатие плотных весов GPU, удаление неиспользуемых форматов (IQ, NVFP4) внутри файлов ядер.

Как запустить

```
tools/setup_venv.sh
tools/build.sh
tools/fetch_model.sh
bin/strata-serve                    # no memory limit (profile auto)
tools/spi                    # pi on the local model, sandboxed to the current folder (doc
```

Все цифры — с ноутбука i7-11370H, RTX 3050 Ti 4 ГБ, 31 ГБ ОЗУ, NVMe; подробности и таблицы в docs/TUNING.md.